

CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU

Bacharelado em Ciência da Computação

GRUPO 18

GROWTH INTELLIGENCE HUB (GIH)

Plataforma de inteligência de crescimento para redes de parceiros
em marketplaces regionais de delivery

DOCUMENTAÇÃO TÉCNICA FINAL

Arquitetura, modelo de dados, decisões e resultados medidos · versão fb4de43, de 28/09/2026

EQUIPE

Ingryd Vitoria de Araújo Barbosa — 01642893

João Pedro Nunes de França — 01626444

Marcio Maycom — 01607574

Pedro Miranda — 01607408

Thiago José Falcão de Freitas — 01597267

Projeto Integrador

Fábrica de Software · Prof.ª Priscilla Gonçalves

Tópicos Avançados · Prof. Antenor Parnaíba

2026.2

Sobre este documento

Este documento reúne a documentação técnica do Growth Intelligence Hub num lugar só: como o sistema está organizado e por quê, como os dados são modelados, como o código se divide em classes e serviços, e o que foi medido — o otimizador em CPU e em GPU, o modelo preditivo, o assistente e a acessibilidade das telas.

Ele não foi escrito à parte. Cada parte é o arquivo correspondente do repositório, convertido como está: a arquitetura e as decisões registradas (ADRs) em docs/07, o modelo de dados em docs/08, as classes e os serviços em docs/10, e as medições em docs/medicoes/. As medições, por sua vez, são geradas por script a cada execução, com o comando e a semente que as reproduzem.

Versão: commit fb4de43 do repositório github.com/PedroMiranda243/gih-fabrica-de-software. **Para regerar:** `node docs/entrega/renderizar_diagramas.js` e `node docs/entrega/gerar_documentacao.js`.

Os requisitos, os casos de uso, o backlog, o cronograma e o processo estão nas entregas da disciplina (docs/entregas/) e em docs/01 a docs/06; o sistema visual, em docs/09.

Sumário

Parte I — Arquitetura e decisões

1. Visão geral
2. Stack
3. Modelo de dados
4. O núcleo computacional
5. Decisões de arquitetura
6. Ambiente de desenvolvimento
7. Estrutura de diretórios
8. Segurança

Parte II — Modelo de dados

1. Modelo conceitual (MER)
2. Modelo relacional
3. Índices
4. Tipos enumerados
5. O que o banco garante sozinho
6. O banco criado — evidência
7. Verificação
8. Onde está a implementação

Parte III — Classes e serviços

1. Visão de integração
2. Domínio — acesso, sessões e auditoria
3. Domínio — parceiros e dados de desempenho
4. Domínio — núcleo computacional e comunicação
5. Camada de serviços — acesso e segurança
6. Camada de serviços — ingestão, análise e comunicação
7. Núcleo computacional — as estruturas
8. Núcleo computacional — as três implementações
9. O que já existe e o que está previsto
10. Como conferir este diagrama contra o código

Parte IV — Resultados medidos

- Medição do núcleo em C++: serial, OpenMP e GPU — H53b, H54a, H54b, H54c
- Medição do otimizador serial — H48, H49
- Medição do modelo preditivo — H42, H43, H46
- Medição do assistente — H65 a H68
- Acessibilidade e responsividade — H76

Parte I — Arquitetura e decisões

Projeto: Growth Intelligence Hub (GIH) **Versão:** 2.0 — 28/09/2026, o sistema construído (1.0 — 03/09/2026, a arquitetura preliminar da Sprint 1)

O sistema como está, e as decisões que o trouxeram até aqui. Este documento começou como a arquitetura preliminar da Sprint 1. Cada decisão tomada na construção entrou como uma ADR, na seção 5, com a data e o que foi medido para tomá-la; as seções 1 a 4 e 6 a 8 descrevem o que foi construído. O arquivo guarda o nome antigo porque os links do repositório e das entregas apontam para ele.

1. Visão geral

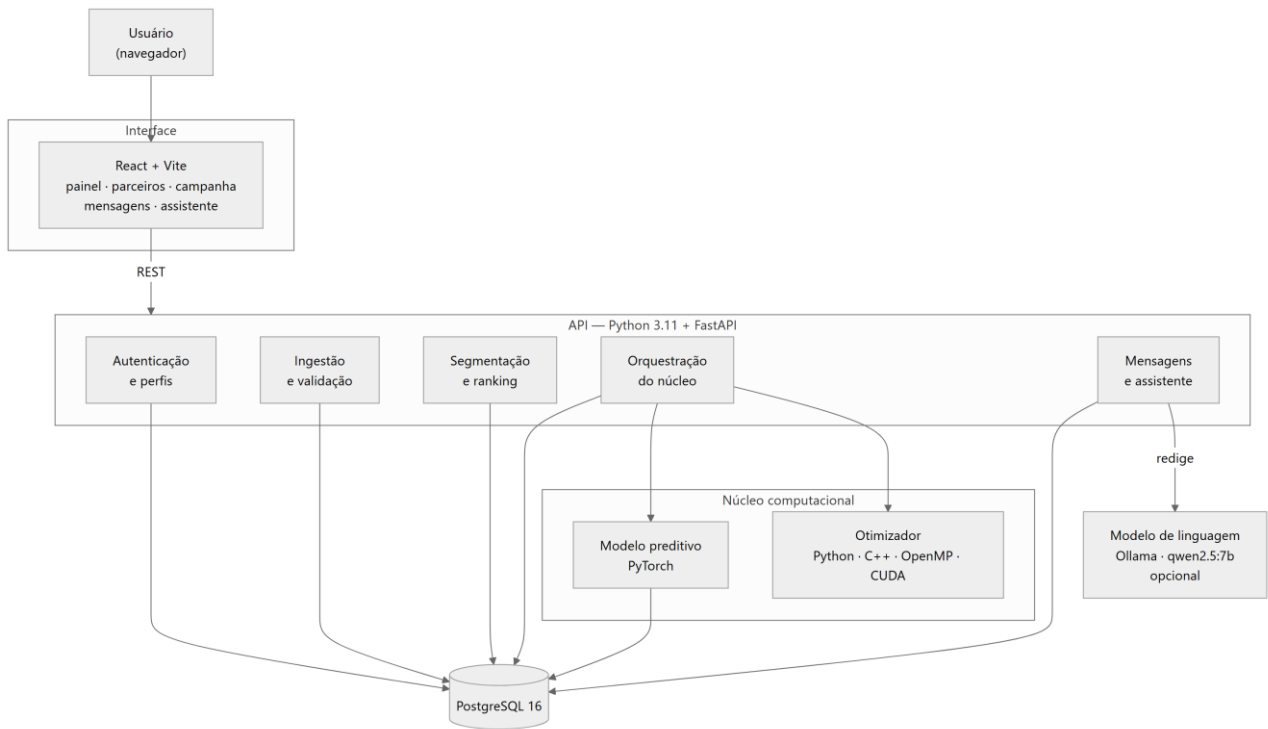


Figura 1 — Arquitetura geral: as camadas e o que cada uma faz

Onde cada parte roda. A interface, a API e o banco são os três contêineres do `docker-compose.yml`. O núcleo não é um serviço à parte: o modelo preditivo roda no processo da API, em segundo plano (ADR-010), e o otimizador em C++ é um executável dentro da imagem dela, chamado por processo (ADR-012). A GPU entra pelo `docker-compose.gpu.yml`, e o modelo de linguagem pelo perfil `assistente`, que sobe o Ollama num contêiner sem porta publicada no host (ADR-013). Sem um e sem o outro, o sistema sobe igual: o otimizador roda na CPU, e as mensagens saem do modelo fixo.

Divisão de responsabilidades

Camada	Responsabilidade	O que não faz
Interface	Exibe, formata e coleta entrada	Nenhuma regra de negócio
API	Regras de negócio, autorização, persistência, orquestração	Não executa cálculo pesado no processo da requisição
Núcleo computacional	Previsão e otimização	Não conhece autenticação nem persistência
Assistente	Redige texto e responde perguntas	Nunca calcula número
Banco	Verdade persistida	Não contém lógica de negócio

Duas consequências práticas dessa divisão:

- A interface pode ser reescrita sem tocar no restante do sistema.

- O núcleo em C++/CUDA é testável isoladamente, por linha de comando, sem subir a API — o que torna o benchmark reproduzível e independente do resto.

2. Stack

Camada	Tecnologia	Justificativa
Núcleo computacional	C++17 + OpenMP + CUDA	Linguagem prioritária da disciplina; controle de memória e paralelismo necessário para o otimizador
Modelo preditivo	Python + PyTorch + NumPy	Modelo treinado pela equipe; PyTorch está na lista recomendada e usa a mesma GPU
API	Python 3.11 + FastAPI	Linguagem prioritária; integra nativamente com PyTorch, e chama o núcleo em C++ como executável (ADR-012); documentação de API gerada automaticamente
Banco	PostgreSQL 16	Relacional, recomendado pela disciplina, adequado a séries por período
Migrações	Alembic	Esquema versionado e reproduzível
Interface	React + Vite	Recomendado pela disciplina; separação clara de responsabilidade no time
Gráficos	Recharts	Integra com React sem dependência pesada
Assistente	LLM local via Ollama (qwen2.5:7b, Apache 2.0)	Executa na máquina; papel restrito à redação, e opcional (ADR-013)
Testes	pytest e Vitest	Cobertura do núcleo e da interface
Empacotamento	Docker Compose	Ambiente reproduzível com um comando
Versionamento	Git + GitHub	Issues, Projects, Pull Requests e Actions

Sobre o JavaScript

A disciplina orienta que JavaScript **não** seja a tecnologia do núcleo computacional. A arquitetura respeita isso integralmente: JavaScript existe apenas na camada de interface. Inteligência artificial, processamento intensivo, paralelização, otimização e GPU estão em Python, C++ e CUDA.

3. Modelo de dados

Este documento trazia um diagrama entidade-relacionamento preliminar, feito antes da modelagem detalhada. Ele foi **removido em vez de atualizado**: existiam duas versões do mesmo diagrama no repositório, e elas já tinham divergido — discordavam da cardinalidade entre `EXECUCAO_OTIMIZADOR` e `PLANO_CAMPANHA`. Duplicata de diagrama sempre diverge, e a que ninguém está olhando é a que fica errada.

O modelo de dados vive em `08-modelo-de-dados.md`, em uma versão só: modelo conceitual, modelo relacional com tipos e chaves, índices, restrições e a evidência do banco criado.

As vinte entidades, em resumo:

Entidade	Papel
USUARIO	Credenciais e perfil de acesso
SESSAO_ACESSO	Sessão autenticada, com estado no servidor
TENTATIVA_LOGIN	Tentativas de autenticação, base do bloqueio por força bruta
AUDITORIA	Trilha de ações sensíveis
CATEGORIA	Classificação do parceiro, com a marca de confirmada ou sugerida
PARCEIRO	Comércio da rede
PERIODO	Janela de tempo de um relatório
IMPORTACAO	Registro de uma carga de dados
METRICA	Faturamento e pedidos de um parceiro em um período

Entidade	Papel
HISTORICO_SEGMENTO	Segmento atribuído a um parceiro em um período
CONFIGURACAO_SEGMENTACAO	Os limiares da RN01, configuráveis sem alterar código (RF21)
PREVISAO	Faturamento previsto e risco estimado
TREINO_MODELO	Uma execução do treino do modelo, com métricas, versão em uso e pesos (RF27)
ACAO_COMERCIAL	Tipo de ação disponível, com custo e efeito esperado
EXECUCAO_OTIMIZADOR	Parâmetros, modo, tempo e resultado de uma execução
EXECUCAO_BENCHMARK	Uma medição dos modos do otimizador lado a lado, com o mesmo plano (H57)
PLANO_CAMPANHA	Solução retornada pelo otimizador
ITEM_PLANO	Par parceiro-ação selecionado
LOTE_MENSAGENS	Uma geração de mensagens em segundo plano: o público, o andamento e as falhas (H60)
MSG	Texto gerado, com estado e decisão humana

Diagrama de classes, incluindo a camada de serviços e o núcleo computacional: `10-diagrama-de-classes.md`.

4. O núcleo computacional

4.1 Formalização do problema

Dado um conjunto de **N** parceiros elegíveis e **A** tipos de ação comercial, escolher no máximo uma ação por parceiro de modo a **maximizar o ganho esperado (uplift)**, respeitando o orçamento, a capacidade e as cotas. Elegível é o parceiro ativo com previsão da versão em uso (RN11).

Elemento	Definição
Decisão	$x[i, a] \in \{0, 1\}$ — o parceiro i recebe a ação a ; $\sum_a x[i, a] \leq 1$
Objetivo	maximizar $\sum u(i, a) \cdot x[i, a]$, com $u(i, a) = \hat{f}_i \cdot c_a + \hat{f}_i \cdot p_i \cdot r_a$ (RN10)
Orçamento	$\sum \text{custo}_a \cdot x[i, a] \leq B$
Capacidade	$n = \sum x[i, a] \leq K$, o número máximo de ações
Cota da categoria k	$\lfloor \min_k \cdot K \rfloor \leq n_k \leq \lfloor \max_k \cdot K \rfloor$, contando só a categoria confirmada (RN11)
Cota da cauda longa	$n_{\text{cauda}} \geq \lfloor m \cdot K \rfloor$, em que a cauda longa é quem está fora do Top N no ranking do período-base

As cotas são frações de **K** e viram contagens antes da busca (RN11): o núcleo recebe inteiros, e não frações. Dinheiro também chega em centavos inteiros (ADR-011).

Viabilidade decidida antes da busca (RF31, RN07). Todo parceiro pode receber a ação mais barata, então basta saber se o **menor conjunto que cumpre os mínimos** cabe em **K** e no orçamento a esse preço. O conjunto se monta assim:

1. Em cada categoria, os mínimos são preenchidos primeiro com parceiros da cauda longa, que contam duas vezes.
2. O que ainda faltar de cauda longa sai das categorias com folga no máximo, ou de quem não tem categoria.

A conta é exata, e a recusa diz qual restrição falhou e quanto falta.

Por que não é trivial: o espaço de busca tem $(A+1)^N$ configurações. Com $N = 200$ e $A = 4$, isso ultrapassa 10^{139} — força bruta está fora de questão, e a estrutura das restrições de cota impede a decomposição gulosa que resolveria o caso sem elas.

4.2 Estratégia de solução

Metaheurística populacional com múltiplas partidas independentes. A escolha é deliberada: partidas independentes são **naturalmente paralelas**, o que torna o problema um caso legítimo — e não artificial — de paralelização em CPU e GPU.

Versão	Tecnologia	Papel
Baseline	Python puro	Referência de corretude e de tempo. Validado contra instância pequena com ótimo conhecido

Versão	Tecnologia	Papel
Serial em C++	C++17	O mesmo algoritmo, linha a linha: a referência contra a qual o paralelismo é medido, para o ganho não medir o compilador junto (H53a)
CPU paralela	C++17 + OpenMP	Os filhos de cada geração, de todas as partidas de uma vez, distribuídos entre os núcleos da CPU (H53b)
GPU	CUDA	Avaliação da população em paralelo massivo na GPU

As quatro versões resolvem o **mesmo problema com a mesma semente**, e é isso que dá sentido à comparação: o *speedup* só é honesto se a qualidade da solução for equivalente (RNF02, tolerância de 2%). O algoritmo, o tratamento das restrições e o que torna as quatro versões idênticas estão na ADR-011.

4.3 Cenário de referência do benchmark

Parâmetro	Valor
Parceiros	2.000
Tipos de ação	5
Repetições	10
Meta de tempo (GPU)	até 5 s
Meta de <i>speedup</i>	no mínimo 5x sobre o baseline serial
Tolerância de qualidade	2% de diferença no uplift

O gerador de dados sintéticos (H28) produz as instâncias. Escala real de operação — cerca de 100 parceiros — é pequena demais para evidenciar ganho de paralelismo: o custo de transferência para a GPU dominaria o tempo total. O cenário ampliado é o que torna a medição significativa, e essa limitação está documentada como parte do resultado.

Medido (26/09/2026, docs/medicoes/nucleo.md): no cenário de referência, a GPU responde em 248 ms de ponta a ponta, com o processo inteiro, contra 26,29 s da busca em Python — **106x**, com o mesmo plano, sem diferença no uplift. As duas metas atendem. Contra o C++ serial, que é a comparação justa do paralelismo, o OpenMP com 8 threads ganha perto de 6x.

4.4 Modelo preditivo

Item	Definição
Alvo	Faturamento do próximo período e probabilidade de queda — de o parceiro estar Em Risco no período seguinte , pelo critério da RN01 (RN09)
Variáveis	Dos últimos 4 períodos do parceiro: faturamento e pedidos, ticket médio, tendência, posição no ranking do período (em percentil), categoria, tempo de casa
Modelo	Rede neural pequena em PyTorch, com duas saídas: o faturamento e o risco
Separação	Por tempo , para não haver vazamento do futuro: o último período testa, o penúltimo valida (parada antecipada e calibração do risco), os anteriores treinam
Baselines	Faturamento: repetir o último período; média móvel dos últimos 4. Risco: a taxa observada no treino, separada por "caiu no último período"
Métricas	MAPE do faturamento contra os baselines; Brier e erro de calibração do risco contra a referência
Mínimos	8 períodos na base para treinar; 4 de histórico para o parceiro receber previsão (RN09)

Se o modelo aprendido não superar os baselines, isso é reportado e o otimizador segue operando com a melhor estimativa disponível. A conclusão negativa, bem medida, é resultado válido.

Uma versão só entra em uso se superar a referência nas duas saídas — MAPE abaixo do melhor baseline e Brier abaixo da referência do risco. Uma versão que acerta o faturamento e erra o risco alimentaria o otimizador com metade da informação pior do que uma conta simples. Onde o código mora, onde o treino roda e onde ficam os pesos: ADR-010.

5. Decisões de arquitetura

ADR-001 — Python + C++/CUDA como stack do núcleo

Situação: definir a linguagem principal do backend e do núcleo pesado.

Alternativas:

Opção	Avaliação
Python (API) + C++/CUDA (núcleo)	Escolhida
Java + Spring Boot	Robusto, mas fora da lista de tecnologias prioritárias da disciplina; integração com PyTorch e CUDA seria indireta
Node.js	JavaScript no núcleo contraria a orientação explícita da disciplina
C++ puro em todo o sistema	Custo de desenvolvimento da API e da integração desproporcional ao prazo

Decisão: Python 3.11 + FastAPI na API; C++17 com OpenMP e CUDA no núcleo; PyTorch no modelo preditivo.

Consequências: alinhamento direto com as tecnologias prioritárias; uma única linguagem para API, IA e scripts, o que reduz a barreira de entrada para a equipe; em contrapartida, exige atenção com o custo de travessia entre Python e C++, resolvido processando em lote em vez de chamada por item.

ADR-002 — Otimização por metaheurística paralela, não por solver exato

Situação: o problema de alocação é combinatório com restrições.

Alternativas:

Opção	Avaliação
Metaheurística com múltiplas partidas	Escolhida — naturalmente paralela, escala bem, aceita restrições arbitrárias
Programação linear inteira com solver pronto	Daria o ótimo garantido, mas o trabalho viraria <i>configurar um solver</i> — não haveria componente computacional próprio para avaliar
Heurística gulosa	Simples demais; não sustenta as restrições de cota nem justifica paralelismo

Decisão: metaheurística populacional com múltiplas partidas independentes, implementada pela equipe nas três versões.

Consequências: não há garantia de ótimo global — mitigado pela validação contra instâncias pequenas de ótimo conhecido; em troca, o projeto ganha um núcleo computacional próprio, mensurável e paralelizável, que é exatamente o que a integração com Tópicos Avançados pede.

ADR-003 — Segmentação e ranking determinísticos, fora do modelo de linguagem

Situação: definir o que o LLM pode e o que não pode fazer.

Decisão: todo cálculo numérico — ranking, segmentação, métricas, previsão e otimização — é determinístico e implementado em código. O modelo de linguagem apenas redige texto e responde sobre dados já recuperados, citando a fonte ou se abstendo.

Consequências: o painel é reproduzível e auditável; a mesma base sempre produz o mesmo resultado; o sistema não fica refém da variabilidade do modelo. Custo: o assistente é menos "esperto" do que um chatbot solto — e isso é intencional.

ADR-004 — Degradação para CPU quando não houver GPU

Situação: o sistema será executado em máquinas diferentes, incluindo a de avaliação.

Decisão: o otimizador detecta os modos disponíveis em tempo de execução e usa o mais rápido possível. Sem GPU compatível, executa em CPU paralela e informa a substituição na interface.

Consequências: o sistema nunca deixa de funcionar por ausência de hardware específico; o benchmark apresenta os modos disponíveis e explica a ausência dos demais.

ADR-005 — CUDA via NVRTC, sem instalar o toolkit no sistema

Status: Superada pela ADR-006 em 15/09/2026 · mantida como registro do que foi medido **Data:** 15/09/2026

Situação: a máquina de desenvolvimento não tem **nenhum compilador C++** — nem MSVC, nem g++, nem clang++ — e não tem o CUDA Toolkit. No Windows, o `nvcc` depende do MSVC como compilador hospedeiro, o que torna o caminho tradicional uma instalação de vários gigabytes antes da primeira linha de código.

Alternativas:

Opção	Avaliação
CuPy com <code>RawKernel</code> (NVRTC)	Escolhida para o spike. Kernel é CUDA C de verdade, compilado em tempo de execução. Runtime e headers vêm por <code>pip</code> , sem instalação de sistema. Licença MIT
MSVC Build Tools + CUDA Toolkit	Caminho tradicional, necessário para <code>nvcc</code> e obrigatório para a H53 (OpenMP em C++). Custo: vários gigabytes e configuração
Numba com destino CUDA	Escreve-se Python anotado, não CUDA C — afasta o projeto da linguagem que a disciplina prioriza
OpenCL	Alternativa aberta registrada no risco R1; só faria sentido se a NVIDIA saísse do caminho

Decisão: o spike usa CuPy com `RawKernel`. O kernel é CUDA C legítimo e roda na GPU — o que retira o risco R1 sem bloquear o projeto numa instalação longa.

Consequências:

- O risco R1 está retirado com evidência medida: *speedup* total de 8,7x, acima da meta de 5x do RNF02
- A escolha **não resolve a H53**, que pede C++ com OpenMP e exige um compilador C++ de qualquer forma
- Descobriu-se que a transferência consome 95% do tempo, o que define a arquitetura da H54c: a população precisa permanecer na GPU entre gerações
- Se o toolchain C++ for instalado, migrar o kernel para compilação por `nvcc` é direto — o código CUDA C é o mesmo. **Foi o que aconteceu, no mesmo dia:** ver ADR-006

Resultado completo em `nucleo/spike/RESULTADO.md`.

ADR-006 — Toolchain nativo: compilar com nvcc e MSVC, não mais por NVRTC

Status: Decidido **Data:** 15/09/2026

Situação: o MSVC Build Tools 2022 e o CUDA Toolkit 13.4 foram instalados na máquina de desenvolvimento, removendo o impedimento que originou a ADR-005. A H53 exige C++ com OpenMP, que o NVRTC não cobre de forma alguma.

Decisão: o núcleo passa a ser **C++ compilado antecipadamente** — `cl` para a versão serial e OpenMP, `nvcc -arch=native` para a versão CUDA. O CuPy sai do caminho de produção e fica apenas no spike original, como registro histórico.

Consequências:

- O caminho de CPU paralela do RNF06 está validado com medição: **~9x com 16 threads**, resultado idêntico ao serial
- O kernel compilado por `nvcc` confere com a CPU com **erro relativo zero** — mesma ordem de soma, mesmo `float32`
- Mediu-se o que a ADR-005 não podia medir: contra **OpenMP**, e não contra o serial, a GPU com transferência a cada geração ganha só **1,1x a 1,3x**, e abaixo de ~4.000 planos **perde**. Sem a transferência, ganha 11x. Isso promove a residência da população na GPU de otimização a **requisito** da H54c
- `nvcc` no Windows usa o `cl` como compilador hospedeiro, então o ambiente do MSVC precisa ser carregado **antes** do CUDA entrar no `PATH`. Está encapsulado em `nucleo/spike/ambiente.bat`

- O caminho do projeto contém acento, o que quebra o encadeamento de comandos no `cmd`. Por isso compilar é sempre via `construir.bat`, nunca chamando `cl` ou `nvcc` soltos
- **Custo assumido:** compilar deixa de ser reprodutível só com `pip`. Quem for mexer no `nucleo/` precisa do Build Tools e do Toolkit instalados. Para o restante da equipe nada muda — `api/`, `web/` e `modelo/` não dependem disso
- A imagem Docker do núcleo ainda não foi construída; empacotar o `nvcc` é trabalho em aberto — *resolvido pelo spike da parte 3 e pela ADR-012, em 26/09/2026*

Resultado completo em `nucleo/spike/RESULTADO.md`, parte 2.

Adendo (26/09/2026, H54b) — o kernel de avaliação:

- **Um bloco por indivíduo, e não uma thread por indivíduo como no spike.** A população de uma geração tem $4 \times 48 = 192$ indivíduos: uma thread para cada ocuparia uma fração da placa. Com um bloco de 256 threads, as threads leem genes vizinhos, e cada uma soma uma fatia dos parceiros.
- **Inteiros, e não float32.** O spike avaliava em `float32` e conferia com "erro relativo zero" porque somava na mesma ordem da CPU. O kernel de verdade soma na ordem em que as threads terminam. Em centavos inteiros, essa ordem não muda nada (ADR-011): o resultado é **igual**, e não próximo — ganho, custo, ações, cauda, contagem por categoria e violação, conferidos indivíduo a indivíduo contra o `avaliar` do Python.
- **A avaliação de um indivíduo é uma função de dispositivo** (`gpu_avaliacao.cuh`), e não só o kernel: o laço da H54c a chama para cada filho, dentro do kernel dele. O tempo medido está em `docs/medicoes/nucleo.md`.

Adendo (26/09/2026, H54c) — o laço inteiro na GPU:

- **Uma geração é um kernel**, com um bloco por indivíduo da geração nova, de todas as partidas juntas: $4 \times 48 = 192$ blocos, como os 188 filhos por geração do OpenMP (adendo H53b da ADR-011). O bloco 0 de cada partida copia a elite. Cada um dos outros faz os dois torneios, monta o filho gene a gene e o avalia com a função da H54b, pelas mesmas threads que o escreveram. Cruzar e avaliar num kernel só evita uma volta à memória global e um lançamento a mais por geração.
- **A população nasce na placa.** A geração 0 é sorteada lá, pelas coordenadas. Da CPU vão só a instância e os dois gulosos, e voltam as avaliações da última geração (40 bytes por indivíduo) e os genes do vencedor, se ele não for um dos gulosos. A população nunca atravessa o barramento — no spike, mandá-la e trazê-la a cada geração era 88% do tempo.
- **O mesmo plano, por construção, e não por cópia.** O gerador, a comparação, o torneio, a elite e a fórmula de cada gene são as mesmas funções da CPU, compiladas para os dois lados (`GIH_CPU_E_GPU`, em `nucleo.hpp` e `genetico.hpp`); não há uma segunda versão em CUDA para divergir. O plano vencedor é avaliado de novo na CPU ao voltar, e diferente seria defeito. Os testes conferem genes, avaliação e gerações contra o Python nas campanhas pequenas, com oito combinações de parâmetros, e contra o C++ serial com 2.000 e 10.000 parceiros; a medição confere em toda execução.
- **O limite de tempo, com a fila cheia.** Os lançamentos são assíncronos: sem espera nenhuma, a CPU enfileiraria as 150 gerações num milissegundo e o limite (UC08-E2) não pararia nada. A CPU deixa até quatro gerações na fila e espera a mais antiga antes de lançar a próxima. A placa não fica parada, e o relógio é olhado com no máximo quatro gerações de atraso.
- **Medido no contêiner, com a placa partindo do repouso a cada busca:** o laço leva **7,3 ms** com 2.000 parceiros, contra 54,5 ms do OpenMP com 8 threads (**7,4x**), e 27 ms com 10.000, contra 251 ms (**9,3x**). O que sobra de custo não é conta: é o contexto da GPU, no adendo H54c da ADR-012.

ADR-007 — Sessão com estado no servidor, não token autocontido

Status: Decidido Data: 15/09/2026

Situação: o RF01 pede sessão identificada, o RF02 pede que o encerramento a invalide **no servidor**, e o RNF10 pede cookie com `HttpOnly` e `SameSite` e renovação do identificador no momento da autenticação.

Alternativas:

Opção	Avaliação
Identificador opaco com estado no servidor	Escolhida. 256 bits sorteados, guardados como hash; a linha em <code>sessao_acesso</code> é a fonte da verdade

Opção	Avaliação
JWT em cookie	Rejeitada. Um token assinado só deixa de valer quando expira, e o RF02 exige efeito imediato. A lista de revogação que contornaria isso reintroduz exatamente o estado que o JWT existia para evitar
Sessão na memória do processo	Rejeitada. Some a cada reinício do contêiner e não sobrevive a mais de uma réplica

Decisão: identificador opaco, `secrets.token_urlsafe(32)`, gravado como SHA-256. Argon2 seria errado aqui: o token já tem entropia suficiente para não haver força bruta a temer, e a conferência acontece a cada requisição, onde um hash lento custaria caro. O que se protege é o vazamento da tabela.

Consequências:

- Encerrar sessão, desativar usuário e trocar senha têm efeito **imediato**, não no fim da sessão em curso
- Uma consulta a mais por requisição, por índice único — aceitável
- **Não existe segredo de assinatura para gerenciar**, um a menos no `.env`
- O CSRF fica coberto por `SameSite=Lax`. Se aparecer um fluxo entre sites, passa a ser preciso um token dedicado
- **Pendente:** a tabela de sessões só cresce. A limpeza das expiradas não foi implementada e vira tarefa quando o volume justificar

ADR-008 — A auditoria grava em transação própria

Status: Decidido **Data:** 15/09/2026

Situação: o RF06 manda registrar as ações sensíveis, e a mais sensível de todas — a tentativa de login recusada — termina numa resposta de erro.

Decisão: `app/auditoria.py` abre a própria sessão de banco e comita imediatamente, separada da transação da requisição.

Consequências:

- Se a auditoria participasse da transação da requisição, o `rollback` da falha levaria junto o registro da falha. A trilha teria **só sucessos** — e mentiria por omissão exatamente sobre o que se investiga
- O mesmo vale para `tentativa_login`: sem transação própria, o contador de força bruta apagaria as próprias evidências e nunca chegaria às cinco falhas do RNF11
- Em troca, existe uma janela em que a operação é revertida e o registro fica. É o lado certo do compromisso: registro a mais é ruído, registro a menos é ponto cego
- Falha ao auditar **nunca** derruba a operação do usuário; vai para o log

ADR-009 — Formato do relatório importado: tolerante, guiado por cabeçalho

Status: Decidido pela equipe **Data:** 16/09/2026

Situação: a H21 pedia "interpreta o formato definido", mas o formato não estava definido em lugar nenhum de `docs/`. O UC03 diz *o que* precisa ser extraído — nome do parceiro, faturamento e número de pedidos — e não *como* o texto chega.

Alternativas:

Opção	Avaliação
Tolerante, guiado por cabeçalho	Escolhida. Separador <code>;</code> , tabulação, <code> </code> ou <code>;</code> ; ordem das colunas livre; sinônimos aceitos nos nomes
Formato fixo e declarado	Interpretador mais simples e erro mais preciso, mas quebra quando a origem muda a exportação, e obriga o usuário a arrumar o texto antes de colar
Texto em prosa, por expressão regular	Serviria se o relatório fosse texto corrido. Mais frágil, e difícil de explicar quando rejeita uma linha

Decisão: o relatório precisa de **uma linha de cabeçalho** nomeando as colunas. Exemplo mínimo:

Parceiro;Faturamento;Pedidos
Comércio Alfa;12500,40;312
Comércio Beta;8940,00;201

Coluna	Sinônimos aceitos
nome	parceiro, nome, estabelecimento, loja, comercio, restaurante
faturamento	faturamento, vendas, valor, receita, total
pedidos	pedidos, qtd, quantidade, numero de pedidos

Acentuação e caixa são ignoradas. Colunas extras são ignoradas. O valor monetário aceita 12.500,40, 12500,40, 12500.40 e R\$ 12.500,40 — **o último separador que aparecer é o decimal**.

Consequências:

- Colar de planilha (que produz tabulação) e importar CSV exportado passam pelo **mesmo** interpretador. A H22, na Sprint 6, vira só a leitura do arquivo e a decodificação
- A vírgula é o **último** separador testado de propósito: em português ela também é decimal, e num arquivo separado por vírgula o valor 1.000,50 seria partido ao meio
- Linha ruim não interrompe a leitura — vira rejeição com motivo, e o usuário decide na prévia se segue com o resto (UC03, A5)
- **Custo assumido:** tolerância esconde erro. Um cabeçalho escrito `Faturamentos` não é reconhecido e o relatório inteiro é recusado. A mitigação é a mensagem do fluxo A4, que devolve as três primeiras linhas recebidas junto com a recusa
- O interpretador é função pura de texto para resultado, em `app/leitor_relatorio.py`. É o que permite a prévia da H24 usar **o mesmo** código da gravação da H21, sem risco de a prévia mostrar uma coisa e a gravação fazer outra

ADR-010 — Modelo preditivo: pacote próprio, treino em segundo plano, pesos no banco

Status: Decidido **Data:** 24/09/2026

Situação: a Sprint 05 da disciplina entrega o módulo de previsão (UC07, RF27, RF28, histórias H41 a H46). Três perguntas não tinham resposta em `docs/`: onde mora o código do modelo; onde o treino roda, já que a API não faz cálculo pesado no processo da requisição (seção 1); e onde ficam os pesos treinados.

Alternativas — onde o treino roda:

Opção	Avaliação
Dentro da requisição	Mais simples, mas segura a conexão enquanto treina, e o navegador desiste antes do fim
Fila de tarefas com serviço próprio (Celery ou RQ, com Redis)	O caminho quando a carga cresce. Aqui traz um serviço e uma dependência a mais para um treino disparado à mão
Tarefa em segundo plano no processo da API, com o estado no banco	Escolhida. A requisição devolve na hora, e a tela acompanha o estado gravado

Alternativas — onde ficam os pesos:

Opção	Avaliação
Arquivo num volume do Docker	Mais um volume para lembrar de preservar, e o banco passa a apontar para algo que pode não existir
Na linha do treino, no banco	Escolhida. A rede tem poucos milhares de parâmetros; os pesos cabem numa coluna

Decisão:

- **modelo/** é um pacote Python próprio, `gih_modelo`: variáveis, baselines, rede, treino e avaliação. Recebe séries numéricas e devolve números. Não conhece banco, FastAPI nem regra de negócio — o mesmo desenho do `nucleo/`, testável por linha de comando sem subir a API.
- **A API orquestra**, em `app/servico_previsao.py`: lê o histórico numa consulta agregada, calcula o rótulo de risco com a mesma função que classifica o segmento (RN09), chama o modelo e grava o treino e as previsões. A regra de negócio fica onde as outras estão.
- **O treino roda em segundo plano**. A requisição devolve 202 com o treino criado, e a tela consulta o estado até ele terminar.
- **Um treino por vez, garantido pelo banco**, com um índice único parcial sobre os treinos em andamento. Uma trava em memória não serviria: com mais de um processo, um não enxerga a memória do outro.
- **Treino interrompido não prende a trava**. Se a API reiniciar no meio de um treino, a subida seguinte o marca como falho, com o motivo. Sem isso, a trava de um por vez ficaria fechada para sempre.
- **Os pesos ficam na linha do treino**, junto com a normalização e o vocabulário de categorias usados. A versão em uso sobrevive a reinício, e uma versão antiga consegue prever sobre dados novos — é o que o UC07-A1 precisa para "manter a versão anterior" depois que chega um período novo.
- **PyTorch na roda CPU** (issue #86). A rede é pequena e a CPU basta; o sistema funciona sem GPU (RNF06), e a GPU fica para o otimizador.
- **Reprodutível** (RNF16): semente fixa, algoritmos determinísticos e uma thread. A mesma base com a mesma semente dá as mesmas métricas — e isso é teste, não promessa.
- **A imagem da API recebe modelo/ como contexto adicional de build**, sem trocar o contexto principal. Construir a partir da raiz do repositório levaria `web/node_modules` e `docs/` para dentro do build.

Consequências:

- Enquanto treina, o treino ocupa um núcleo do processo da API. O tempo é medido e registrado junto com as métricas (H46); se a base crescer a ponto de pesar, o treino vira um processo separado **sem mudar o contrato da tela**, porque a tela só conhece o estado gravado
- A versão em uso é sempre derivável do banco: cada treino concluído registra qual versão ficou em uso depois dele — a própria, se superou a referência, ou a anterior, com o motivo
- **Custo assumido**: o PyTorch aumenta a imagem da API e o tempo da CI. A roda CPU é a menor disponível, e o tamanho medido fica registrado no Pull Request que a introduz

ADR-011 — Otimizador: genético com partidas independentes, restrições por viabilidade, aritmética inteira

Status: Decidido Data: 25/09/2026

Situação: a ADR-002 fixou a família do algoritmo, uma metaheurística populacional com partidas independentes. Faltava decidir três coisas antes da primeira linha (H48, H49):

- como as restrições entram na busca;
- o que garante que o plano devolvido seja viável (RN07);
- como três implementações (Python, C++ com OpenMP e CUDA) chegam ao mesmo resultado. A H53a pede resultado idêntico ao do Python, e o RNF02 mede o *speedup* com qualidade equivalente.

Alternativas — restrições:

Opção	Avaliação
Penalidade somada ao objetivo	O peso da penalidade é arbitrário, e com o peso errado o melhor encontrado é inviável
Decodificador que só gera soluções viáveis	Garante viabilidade, mas a construção fixa escolhas (quem cobre a cota) e pode deixar o ótimo fora do alcance da busca
Comparação por viabilidade, com semente viável e elitismo	Escolhida. Viável vence inviável; entre viáveis, maior ganho; entre inviáveis, menor violação. Sem peso para calibrar, e a busca pode atravessar o inviável para chegar ao ótimo

Decisão:

- **Algoritmo genético, com partidas independentes** e o melhor delas ao final.

- Cada solução é um vetor com um gene por parceiro: 0 é "sem ação", e de 1 a A é a ação.
- Seleção por torneio de dois, cruzamento uniforme e mutação que sorteia outro valor para o gene.
- Cada geração guarda o melhor da anterior (elitismo).
- **As duas primeiras soluções de toda partida são viáveis por construção.** Saem da mesma montagem que decide a viabilidade (§4.1): os mínimos com a ação mais barata, completados por um guloso. Uma completa pela razão ganho/custo, que acerta quando o orçamento aperta; a outra pelo ganho, que acerta quando o que aperta é o máximo de ações. Com o elitismo, o melhor encontrado nunca é pior que a melhor delas. Por isso, **quando a campanha é viável, o plano devolvido é viável**; e ele ainda passa por um verificador independente antes de ser gravado.
- **Uma mutação por filho, em média**, qualquer que seja N. Com duas ou quatro, o genético não saiu dos planos gulosos em duas de três campanhas de teste, e uma mutação por troca (tirar a ação de um parceiro e dar a outro) também rendeu menos. Os números estão na medição do otimizador.
- **A violação é inteira e em unidades de ação:** o excesso sobre K e sobre os máximos, o que falta nos mínimos, e o excesso de orçamento dividido pelo custo da ação mais barata (arredondado para cima). Não há mistura de reais com contagens. Por isso toda ação do catálogo que entra na campanha custa mais que zero: uma ação gratuita não consome orçamento e não é o que uma campanha distribui.
- **Dinheiro em centavos inteiros.** Ganho, custo e orçamento chegam ao núcleo como inteiros. A soma de inteiros dá o mesmo resultado em qualquer ordem, e a de `float` não; sem isso, a soma em paralelo da GPU divergiria da serial no último centavo e mudaria desempates.
- **Gerador aleatório sem estado.** Cada sorteio é o SplitMix64 aplicado em cadeia a (semente, partida, geração, indivíduo, gene). Escrito igual em Python, C++ e CUDA, ele dá o mesmo número em qualquer versão, e cada thread da GPU calcula o seu sem compartilhar estado. Probabilidades são inteiras, em partes por milhão.
- **Gerações síncronas e desempate fixo** (o menor índice vence). Cada filho depende só da geração anterior e do sorteio dele. Serial, OpenMP e GPU executam a mesma sequência, e o paralelismo muda o tempo, não o plano.
- **O baseline serial é Python puro, sem NumPy no laço.** Ele é a referência de corretude e o denominador do *speedup* (RNF02). Vetorizá-lo deixaria o baseline mais rápido e o ganho medido menos honesto.
- **Limite de tempo** (UC08, E2): ao estourar, devolve o melhor viável encontrado e marca o plano como parcial.

Consequências:

- A qualidade é conferida contra a **enumeração exata** em instâncias pequenas, sem solver pronto (ADR-002), e contra o guloso numa instância em que ele falha
- A GPU avalia a população inteira em paralelo, que é o kernel medido no spike da H47, e com a população residente entre gerações (ADR-006)
- Aritmética inteira limita o ganho de um plano a 2⁶³ centavos, muito acima de qualquer campanha
- **Custo assumido:** o baseline em Python puro é lento de propósito. Na tela, o limite de tempo o protege até as versões em C++ chegarem (Sprint 10)

Adendo (26/09/2026, H53b) — o que o OpenMP paraleliza:

- **As partidas avançam juntas, uma geração por vez**, e os filhos de todas elas formam um único laço paralelo: 4 × 47 = 188 tarefas por geração, com os parâmetros padrão. Paralelizar só as partidas, como a §4.2 dizia antes, ocuparia no máximo 4 threads.
- **O plano é o mesmo da versão serial com qualquer número de threads.** Nenhum filho lê o que outro escreve na mesma geração, e o vencedor sai das partidas percorridas na ordem, com a mesma comparação estrita. Os testes conferem com 1, 2, 3 e 7 threads e com o padrão; a medição confere em toda execução.
- **Escalonamento dinâmico, um filho por vez.** Cada geração termina numa barreira, e na divisão estática ela espera a thread mais atrasada. No WSL2, os processadores virtuais são divididos com o Windows, e em 150 gerações sempre há um que atrasa: medido no contêiner, o estático perdeu em todos os tamanhos.
- **O limite de tempo é a única diferença visível.** Na serial, a busca interrompida encerra a partida em curso e não começa as seguintes; no OpenMP, todas param na mesma geração. As duas devolvem o melhor plano viável encontrado e o marcam como parcial.
- **Tantas threads quantos núcleos físicos.** No contêiner, com 2.000 parceiros, duas medições: Um número que muda assim de uma medição para a outra não vai para a frente da banca. A espera passiva nas barreiras tira o pico, mas deixa tudo mais lento. A H55 faz a API pedir o número de núcleos físicos. Medição mais recente em `docs/medicoes/nucleo.md`.

- com 8 threads, o ganho sobre o C++ serial foi de **6,0x** numa e **6,3x** na outra, sempre em faixa estreita (5,3x a 7,0x, somadas as duas);
- com 16, todas as threads lógicas da máquina, a mediana foi de 4,6x numa e 7,2x na outra, e a faixa foi de 1,0x a 8,7x. O contêiner passa a disputar a CPU com o próprio Windows.

ADR-012 — O núcleo é um executável dentro da imagem da API, chamado por processo

Status: Decidido Data: 26/09/2026

Situação: a API roda num contêiner Linux, e o núcleo em C++ com OpenMP e CUDA (H53, H54) só tinha sido compilado no Windows (ADR-006). A Sprint 10 precisa que a API chame o C++. Antes de escrever o porte, o spike da issue #123 mediu se o mesmo código compila em Linux e roda **dentro da imagem da API**, com a GPU (nucleo/spike/RESULTADO.md, parte 3).

Alternativas — onde o núcleo roda:

Opção	Avaliação
Na imagem da API, como executável	Escolhida. Medido: roda em <code>python:3.11-slim</code> com <code>--gpus all</code> , acrescenta ~1 MB à imagem e recusa de forma limpa sem GPU
Serviço próprio no Compose, com a imagem da NVIDIA	Mais um serviço, mais uma chamada pela rede, e 5,1 GB de imagem para bibliotecas que o núcleo não usa
Nativo no host Windows	Sai do Docker: quebra o "sobe com um comando" (RNF07) e depende de a máquina da avaliação ter o MSVC e o toolkit

Alternativas — como a API chama:

Opção	Avaliação
Executável, com a instância na entrada padrão e o plano na saída	Escolhida. A ida e volta do cenário de referência (2.000 parceiros, 76 KB de texto) mediu cerca de 2 ms. Uma falha na GPU derruba o processo do núcleo, e não o da API
Módulo de extensão (pybind11, BSD)	Chamada sem cópia, mas amarra o binário à versão do Python e põe a GPU dentro do processo da API: um erro no kernel derruba o servidor inteiro

Decisão:

- **O C++ vira um executável, compilado num estágio do Dockerfile da API** e copiado para a imagem final, com o runtime do CUDA embutido (`cudart` estático, o padrão do `nvcc`). A imagem da API continua sendo a `python:3.11-slim`; nenhuma camada da NVIDIA vai para ela.
- **A instância vai em texto, em inteiros** — os mesmos centavos e contagens da ADR-011 —, e o plano volta em texto. Lido com a biblioteca padrão do C++: nenhuma dependência nova.
- **O CUDA é opcional no build.** O estágio de CPU (g++ com OpenMP) roda sempre; o de CUDA, só quando um arquivo à parte do Compose pede — e é esse arquivo que reserva a GPU. Sem ele, `docker compose up` não baixa os GB da imagem de compilação da NVIDIA nem exige placa NVIDIA: o sistema sobe em qualquer máquina, em CPU paralela (RNF06, H72).
- **A API pergunta ao executável quais modos existem** (`gih-nucleo versao`). Sem GPU, o CUDA recusa com saída 1, e a API cai para CPU paralela com aviso (ADR-004, H56). Desde a H55, a imagem da API compila o executável num estágio com g++ e OpenMP, na mesma base `python:3.11-slim`. O modo pedido que não existe roda no mais rápido disponível, e a execução diz a troca.
- **O baseline serial em Python continua na API:** é o denominador do *speedup* do RNF02. O serial em C++, o OpenMP e o CUDA ficam no executável, com o mesmo algoritmo e o mesmo gerador aleatório (ADR-011).
- **O benchmark é medido dentro do contêiner**, onde o sistema roda, e não no Windows.
- **O benchmark mostra quatro colunas**, e não três: Python (o baseline do RNF02), C++ serial, OpenMP e CUDA. O porte da H53a mediu o C++ serial **76x** mais rápido que o Python, com o mesmo plano (0,34 s contra 25,5 s, 2.000 parceiros, no contêiner): a meta de 5x do RNF02 já cai sozinha com a compilação. O ganho do **paralelismo** só se lê contra o C++ serial — anunciar o ganho da GPU sobre o Python seria creditar a ela o que é do compilador.

Consequências:

- Uma falha do núcleo volta à API como saída diferente de zero e vira execução **FALHOU** com motivo, como no treino do modelo — a API continua de pé
- No WSL2, a transferência para a GPU custa ~40% a mais que no Windows nativo (parte 3). É mais um motivo para a população ficar residente na GPU entre gerações (H54c, ADR-006)
- O OpenMP do GCC ganha menos que o do MSVC nos tamanhos pequenos (0,8x contra 10,4x em 256 planos). O benchmark vai mostrar isso, e não esconder
- O `construir.bat` continua sendo o caminho para desenvolver o kernel no Windows; o contêiner é a referência para medir e para rodar o sistema
- **Custo assumido:** o build com CUDA baixa a imagem de compilação da NVIDIA, de alguns GB, sob licença proprietária — a mesma exceção do CUDA Toolkit na regra 2.8 do `CLAUDE.md`

Adendo (26/09/2026, H54a) — a GPU no executável:

- **O CUDA fica nos arquivos .cu**, compilados quando há toolkit: pelo `construir.bat`, se acha o `nvcc`, e pela imagem do núcleo. Sem ele, `sem_gpu.cpp` responde que não há GPU. A CI compila só os `.cpp`: não tem placa nem toolkit.
- **A saída 1 passa a ter dono.** É "o pedido precisa de GPU, e não há" — executável sem CUDA, nenhuma placa visível, driver antigo —, com o motivo no erro padrão. `gih-nucleo versao` diz a GPU ou o motivo, e `gih-nucleo gpu` recusa com a saída 1: é o que a API pergunta (H56).
- **As estruturas na GPU seguem o layout da CPU:** a tabela de ganho achatada, em centavos, e os genes de um byte. A população tem dois buffers, a geração atual e a seguinte, trocados por ponteiro, que ficam na placa a busca inteira (ADR-006, H54c).
- **O modo `cuda` só aparece nos modos do executável com a H54c**, quando `otimizar` rodar na GPU. Até lá, a campanha continua mostrando a GPU como indisponível. A transferência medida está em `docs/medicoes/nucleo.md`.

Adendo (26/09/2026, H56) — sem GPU, o sistema funciona igual:

- **O arquivo à parte existe: `docker-compose.gpu.yml`.** Ele passa `NUCLEO=cuda` ao build da API, que compila o núcleo no estágio da imagem da NVIDIA, e reserva a placa. Sem ele, o estágio de CPU é o único construído, e a imagem da NVIDIA nem é baixada.
- **O executável diz por que não há GPU num código:** `sem_cuda`, `sem_placa` ou `erro`. A API o traduz para a tela, e a mensagem do runtime do CUDA fica para quem investiga. Com a placa presente e o modo `cuda` ainda fora do executável, a tela diz isso também: a GPU está lá, e o otimizador nela é da H54c.
- **A GPU que falha no meio do cálculo cai para a CPU.** O modo é escolhido quando a GPU responde. Se o executável sai com 1 no cálculo, a busca roda de novo no mais rápido que sobrou, e a execução passa a dizer esse modo e a troca (UC08-A4). Os modos dão o mesmo plano (ADR-011): o gestor recebe o mesmo resultado, mais devagar.
- Conferido em contêiner de verdade, nos três casos: build padrão (`sem_cuda`), com o arquivo da GPU (a RTX 4060 aparece na API) e a mesma imagem sem a reserva (`sem_placa`).

Adendo (26/09/2026, H54c) — o modo `cuda`, e o custo fixo da GPU:

- **O modo `cuda` está nos modos de todo executável compilado com CUDA;** se há placa, quem diz é a linha da GPU. A API oferece a GPU só com as duas coisas: a imagem com CUDA num contêiner sem a reserva da placa mostra a GPU indisponível, com "nenhuma GPU NVIDIA disponível nesta máquina". A frase do adendo H56 para a placa presente sem o modo deixou de existir.
- **A campanha inviável é respondida antes, na CPU, em qualquer modo** — inclusive no `cuda` sem placa. A viabilidade não precisa de GPU, e a resposta é a mesma.
- **A placa que falta no meio da busca também é saída 1:** memória esgotada, driver caído, placa ocupada ou sumida. A API calcula na CPU, com o mesmo plano (adendo H56). Um erro dentro do kernel continua sendo defeito, com a saída 3.
- **A GPU tem um custo fixo: ~0,2 s por cálculo.** Iniciar o driver e criar o contexto da placa custa, no contêiner, de 170 a 230 ms, com 500 ou com 10.000 parceiros, antes de qualquer conta; no Windows, menos da metade. O executável o informa à parte, e a medição mostra o laço sem ele e a busca inteira com ele. Com 2.000 parceiros, a busca inteira na GPU leva 195 ms, contra 54,5 ms do OpenMP: o laço ganha 7,4x, e o contexto devolve tudo. Com 10.000, a GPU já termina antes (1,1x).

- **O automático continua preferindo a GPU.** A diferença, nas campanhas de poucos milhares de parceiros, é de décimos de segundo, que a tela não sente. Em troca, a tela deixou de prometer "o mais rápido": diz "na GPU, se houver, e senão no CPU paralelo", e a descrição do modo GPU avisa que ele compensa nas campanhas grandes. A leitura da RF32 está numa nota do UC08, em [docs/03](#).
- **Um processo por cálculo continua valendo.** Um núcleo residente, com o contexto criado uma vez, tiraria o custo fixo, mas traria de volta o que esta ADR evitou: um processo de vida longa para a API gerenciar e reerguer. Com o RNF01 e o RNF02 atendidos com folga, fica registrado como caminho, e não como pendência.
- **RNF01 e RNF02, no cenário de referência** (2.000 parceiros e 5 ações, no contêiner): o processo inteiro na GPU, da chamada à resposta, leva **248 ms**, contra a meta de 5 s. O Python leva 26,3 s, **106x** mais, e dá o mesmo plano: 0,0% de diferença no uplift, contra os 2% permitidos. Números em [docs/medicoes/nucleo.md](#).

Adendo (27/09/2026, H57) — o benchmark na tela:

- **As quatro colunas desta ADR estão na tela** ([/benchmark](#), UC09): o mesmo problema sintético, com a mesma semente, no Python e nos três modos do executável. O ganho do OpenMP e o da GPU aparecem também contra o C++ serial.
- **O Python roda num processo à parte**, criado do zero (`spawn`), e não na thread de fundo da API. Lá ele dividiria o interpretador com as requisições da própria tela, que consulta o andamento, e o baseline sairia mais lento — o ganho medido, mais bonito do que é. Criado do zero porque `fork` copiaria a API, com as threads dela.
- **O benchmark é um por vez**, como o otimizador, e **não trava a campanha**: se uma otimização ou um treino roda durante a medição, o resultado sai marcado como disputado, e a tela diz que os tempos podem ter saído maiores.
- **Medido no contêiner, pela própria tela**, com a GPU (`docker-compose.gpu.yml`), e o plano igual nos quatro modos: com 2.000 parceiros, o Python levou 25,5 s, o C++ serial 314 ms, o OpenMP 53 ms e a GPU 228 ms — 112x o Python; com 10.000, a GPU (245 ms) passou o OpenMP (256 ms), como o adendo H54c previa.

ADR-013 — O modelo de linguagem é local, opcional e não escreve número sem origem

Status: Decidido **Data:** 27/09/2026

Situação: a central de comunicação (H60 a H64) redige mensagens, e o assistente (H65 a H68) responde a perguntas. Os dois precisam de um modelo de linguagem, e a regra 2.8 o quer local, pelo Ollama. Três perguntas precisavam de resposta antes da primeira tela:

- onde o modelo roda;
- o que o sistema faz sem ele;
- como se garante que ele não inventa número (RN08). Instrução no texto do pedido não garante.

Uma sonda contra o `qwen2.5:7b`, na RTX 4060, mediu o que segue (issue #154).

Alternativas — o modelo:

Opção	Avaliação
<code>qwen2.5:7b</code>	Escolhido. Apache 2.0. Ocupa 4,4 GiB da placa de 8 GB, e o otimizador cabe junto. Com o modelo carregado, de 2 a 4 s por mensagem. A saída por esquema JSON funciona
<code>qwen2.5:14b</code>	9 GB: não cabe na placa, e uma parte rodaria na CPU
llama3 (8B)	A licença da Meta restringe o uso: não é MIT, Apache nem BSD (regra 2.8)

Alternativas — onde roda:

Opção	Avaliação
No Compose, num perfil (assistente)	Escolhida. Opcional como a GPU: sem o perfil, a imagem nem é baixada. Com o <code>docker-compose.gpu.yml</code> , a placa é reservada também para ele
Num arquivo à parte, como o da GPU	Era o plano. A reserva da placa não pode ser condicional num arquivo só: o arquivo do assistente ou exigiria placa NVIDIA, ou nunca a usaria. Com o perfil, o arquivo da GPU cobre os dois
Só o Ollama instalado na máquina	Sai do "sobe com um comando" (RNF07). Continua possível no desenvolvimento, por <code>OLLAMA_BASE_URL</code>

Decisão:

- **O modelo é opcional, e o sistema não depende dele** (RNF06, H72). O endereço vem de `OLLAMA_BASE_URL`. Sem resposta ou sem o modelo baixado, o redator diz por quê (`estado()`), e quem chama segue sem ele:
 - as mensagens saem do **modelo fixo**: textos por segmento, escritos pela equipe, com os mesmos fatos. A mensagem registra que saiu sem o assistente;
 - o assistente se declara indisponível (UC12-E1), e o painel segue.
- **O modelo faz duas coisas, e só elas** (`app/redator.py`):
 - **redigir**: texto em volta de fatos já apurados pelo código;
 - **extrair**: os campos de uma pergunta, num JSON com esquema que o código valida. Quem responde é o código.
- **A guarda numérica é a RN08 como código** (`app/guarda_numerica.py`). Ela lê o texto pronto e aponta todo número que os fatos não sustentam: A comparação é exata: 13% não é 12,8%. Texto com número sem origem não chega à tela: sai o modelo fixo, com o motivo registrado.
 - algarismos, reais, percentuais, datas e ordinais;
 - número por extenso;
 - comparações ("dobro", "metade"), que são conta.
- **Temperatura zero e semente fixa**, para o texto variar o mínimo. Não é garantia: no contêiner, duas chamadas seguidas com os mesmos fatos deram frases diferentes. Nada depende de o texto se repetir; o que se repete são os números, e quem garante é a guarda.
- **Nenhum envio**. Aprovada, a mensagem fica "pronta para envio", e a tela oferece copiar e exportar. Enviar por e-mail ou WhatsApp é serviço externo: fora do escopo e da regra 2.8.
- **A porta do Ollama não é publicada no host**. Ele não tem autenticação; a API o alcança pela rede do Compose.
- **O modelo fica carregado pelo tempo padrão do Ollama, 5 min**. O plano era descarregá-lo logo, para não disputar a memória da placa com o otimizador. A medição desfez o motivo: o modelo ocupa 4,4 GiB, e o otimizador, poucos MB. A disputa que importa é de processamento, durante o benchmark. Por isso a geração de mensagens em curso vai marcar a medição como disputada, como a otimização já marca (H57, H60).

Consequências:

- **A primeira chamada paga o carregamento**. Ela levou 47 s no Ollama do Windows e 42 s no contêiner, com a placa reservada; 25 s são para levar o modelo do disco à placa. Carregado, o modelo leva de 2 a 4 s por mensagem. O limite de tempo do redator é de 120 s, e a tela precisa dizer que o modelo está sendo carregado.
- **O modelo não se contém sozinho — medido, e não suposto**. Na sonda:
 - **A mensagem contou ao parceiro o segmento interno** ("no segmento Em Risco"). A classificação é da rede, e não do parceiro (RF26): o segmento escolhe o tom, mas não entra nos fatos da mensagem.
 - **"Qual a capital da França?" saiu classificada como pergunta de segmento**, com o parceiro "catálogo_geográfico". Quem decide se a pergunta está no catálogo é o código: o parceiro precisa existir, pela busca normalizada (RF24), e, se não existe, a resposta é a abstenção (H68).
 - **A mensagem de um parceiro sem a variação nos fatos disse "foi um bom desempenho"**: um juízo que os fatos não sustentam, e que a guarda não pega, porque não é número. É para isso que nenhuma mensagem sai sem a aprovação de um gestor (RN06): a guarda protege o número, e a pessoa protege o sentido.
- Sem placa, o modelo roda na CPU, bem mais devagar. Na máquina de avaliação sem GPU, as mensagens saem do modelo fixo, e o sistema faz o que o backlog pede.
- **Custo assumido**: a imagem do Ollama (MIT, fixada na 0.32.5) tem ~8 GB, e o modelo ~4,7 GB, baixados só com o perfil.
- A CI não tem o modelo. Os testes usam um Ollama de mentira, e os que falam com o de verdade pulam — exceto com `GIH_ASSISTENTE_OBRIGATORIO=1`, como a GPU.

Adendo (27/09/2026, H60) — as mensagens, contra o modelo de verdade:

- **A instrução foi ajustada contra uma amostra**, e não pela intuição. Com os fatos reais da base, a primeira versão escreveu em terceira pessoa ("o faturamento da Esquina da Serra caiu"), afirmou o que os fatos não

dizem ("bateu novo recorde"), trocou o sentido de um número ("seus pedidos somaram R\$ 20.358,11") e assinou como carta. Cada frase da instrução em `servico_mensagens.SISTEMA` responde a um desses. O que sobra — quebra de linha e "Atenciosamente" — o código tira.

- **A guarda aprendeu a data sem o ano.** Na verificação de ponta a ponta, 2 das 23 mensagens do plano caíram para o modelo fixo porque o modelo escreveu o período da ação como "05/10 a 11/10", e a guarda lia os dias e os meses como números soltos; na rodada seguinte, 1 caiu por "de 05 a 11/10". Hoje ela lê 05/10, de 05 a 11/10, 14 de setembro e de 14 a 20 de setembro como datas, e as confere contra as dos fatos.
- **O juízo continua passando.** "Parabéns pelo bom momento" num parceiro em ascensão é o tom pedido; "excelente desempenho" num parceiro estável não é, e nenhuma guarda o pega, porque não é número. É o que a aprovação humana existe para pegar (RN06, H62).
- **Medido pela verificação de ponta a ponta**, com o Ollama do Windows e o modelo já carregado: as 23 mensagens de um plano em 60 s, todas redigidas pelo modelo, e nenhuma com número fora dos fatos. Sem o modelo, as mesmas 23 saem do modelo fixo na hora.

Adendo (27/09/2026, H65) — o catálogo do assistente, contra o modelo de verdade:

- **O modelo só lê a pergunta.** Ele devolve o tipo, um dos onze do catálogo ou `fora_do_catálogo`, e os campos: o parceiro, a categoria, o segmento, as datas e o tamanho da lista (`app/assistente/catalogo.py`). Quem busca os valores é o código, com as funções das telas; quem escreve a resposta, nesta história, também. Cada resposta traz os fatos que a sustentam, e um teste confere, tipo a tipo, que todo número do texto está neles.
- **O parceiro precisa estar escrito na pergunta**, palavra a palavra, e existir na base. É a resposta ao "catálogo_geográfico" da sonda.
- **Uma sonda de 30 perguntas** mediu a extração. A primeira instrução acertou 19; os erros tinham causa: Com os ajustes, 26 de 30, e uma das quatro que faltam, a do artigo, o código resolve. As três restantes são a média das pizzarias, a comparação entre dois parceiros e o total de um mês. Nenhuma recebe número inventado: viram pedido de precisão, ou o desempenho de um parceiro só. Mas também não recebem a abstenção que merecem, e a instrução não bastou. É a H68, e a solução vai ser por código.
 - **Nenhuma data chegava.** Com os campos opcionais no esquema, o modelo fechava o JSON logo depois do tipo. Todo campo passou a ser obrigatório, ainda que nulo, e a instrução ganhou as expressões de tempo com as datas da base: "semana passada" depende de qual é a semana mais recente.
 - **Uma data saiu `log(2026-09-07)`.** O padrão `AAAA-MM-DD` foi para o esquema, e a gramática da saída passou a impedir outra forma.
 - **O modelo copiou o artigo** ("A Esquina da Serra"), e a busca não achava o nome. A resolução tira o artigo do começo.
 - **"Top 3 farmácias" virou o segmento Top**, e "o faturamento da rede", pergunta de um parceiro. Viraram regras na instrução.
- **Com o modelo carregado, a extração leva cerca de 4 s por pergunta**, no Ollama do Windows.

Adendo (27/09/2026, H66 a H68) — a fonte, a redação e a abstenção, medidas:

- **A fonte é do código** (H66). Cada resposta diz de que períodos consultou os dados, e a rota os devolve como relatórios: o período, quando e por quem foi importado, e a origem. Na previsão e no plano, a fonte traz também a versão do modelo e o cálculo. O modelo de linguagem não vê a fonte, e não tem como citar outra.
- **O modelo redige só a resposta em prosa** (H67): o desempenho, a posição, o segmento, a previsão, o resumo e o plano. O ranking, a lista de um segmento, a evolução, a distribuição e a mobilidade saem como o código as montou. Uma lista reescrita pode perder um item sem que número nenhum fique errado — omissão que a guarda não pega.
- **Três guardas conferem o texto do modelo.** Qualquer uma que reprove, e a resposta é a do código, com o motivo:
 - o número sem origem, a guarda de sempre;
 - **o sentido trocado** (`guarda_numerica.sentido_trocado`, que as mensagens também passaram a usar). A guarda do valor ignora o sinal, porque "caiu 12,50%" é o jeito certo de escrever -12,50%, e por isso deixava passar "caiu 12,50%" onde o fato é +12,50%;
 - **a unidade sem origem** — mês, ano, dia, dólar — que os fatos não têm.
- **A abstenção é do código** (H68), antes do modelo, quando a pergunta pede: Depois do modelo, a pergunta com dois parceiros é comparação, e a previsão de outro período que não o seguinte ao treino também recebe a abstenção.

- o uma conta: média, soma, diferença, acumulado, "desde o começo";
- o uma causa: "por que caiu?";
- o um dado que o relatório não traz: lucro, telefone, estoque.
- **O código também acerta a leitura do modelo, com regra medida** (`servico._corrigir`):
 - o o resumo da rede com um parceiro citado é o desempenho dele;
 - o o desempenho de várias semanas é a evolução semana a semana — ou a abstenção, se a pergunta pede o total;
 - o a data que o modelo não trouxe, o código lê da pergunta: o mês, o ano, "o mês que vem".
- **Medido** (`docs/medicoes/assistente.md`, `scripts/medir_assistente.py`), com três conjuntos versionados em `api/tests/assistente/`: Com o modelo carregado, a extração leva 3,8 s, e a pergunta inteira, com a redação, 9,4 s (medianas).

Conjunto	Resultado
Referência (RF41), 49 perguntas	O tipo e os campos certos em 47. As duas leituras erradas, o código corrige antes de responder
Armadilhas (H67), 18 perguntas	Zero número sem origem na resposta. O modelo redigiu 4 respostas que a guarda aprovou; 11 redações foram recusadas
Sem resposta (H68), 20 perguntas	Abstenção em 20

- **A primeira rodada mudou o código.** A abstenção ficou em 18 de 20: "em 2019" e "em dezembro" chegaram sem data, e a resposta saiu da semana mais recente — era outra pergunta. Duas redações passaram pela guarda com o número certo e a unidade errada: "o faturamento previsto para o mês" (a previsão é de uma semana) e "o ticket médio em dólares foi de R\$ 91,25". Daí a leitura das datas pelo código, o período previsto nos fatos da previsão e a guarda da unidade.
- **A placa cheia derruba o modelo.** Na primeira rodada, com a GPU de 8 GB em 7,8 GB, um pedido trivial levou 114 s em vez de 2,5 s, e perguntas passaram do limite de 180 s. Descarregar e recarregar o modelo resolveu. O assistente não trava — o limite vira "indisponível" —, mas o sintoma é esse.
- **O limite, dito.** As listas de palavras da abstenção cobrem o que foi medido, e uma forma nova de pedir uma conta pode escapar delas. Aí sobram a classificação do modelo e a guarda: o número não sai errado, mas a resposta pode ser de outra pergunta. O juízo também passa: "a chance de cair" onde o fato é "a chance de entrar em risco". É o limite de um assistente sem aprovação humana, e é por isso que ele responde só o catálogo.

6. Ambiente de desenvolvimento

Item	Situação
GPU de desenvolvimento	NVIDIA GeForce RTX 4060, 8 GB, driver 616.56 — disponível e verificada
CUDA Toolkit (<code>nvcc</code>)	13.4 — instalado e validado em 15/09/2026 (H47)
Compilador C++	MSVC 19.44 (Build Tools 2022) — instalado e validado; exigido só para <code>nucleo/</code>
Python	3.11
Node.js	20
Docker Desktop	Necessário para o banco e para o empacotamento

Nem todas as estações da equipe têm GPU. É exatamente por isso que a ADR-004 existe: o desenvolvimento do restante do sistema não pode ficar bloqueado pelo hardware de um integrante.

7. Estrutura de diretórios

```

growth-intelligence-hub/
├── api/
│   └── app/
│       ├── rotas/
│       └── assistente/
└── # FastAPI
    # regras de negócio, serviços e segurança
    # uma rota por recurso
    # catálogo, resolução, respostas, fonte e redação
  
```

└─ migrations/	# Alembic, uma migração por PR
└─ tests/	# pytest, contra um banco de teste
└─ e2e/	# a verificação de ponta a ponta, contra o sistema no ar
└─ nucleo/	# o otimizador
└─ gih_nucleo/	# o baseline em Python, e a chamada ao executável (nativo.py)
└─ cpp/	# serial, OpenMP e CUDA; um executável só
└─ spike/	# o spike de GPU da Sprint 3 (H47), guardado como evidência
└─ tests/	
└─ modelo/	# o modelo preditivo em PyTorch: variáveis, rede, treino, avaliação
└─ web/src/	# React + Vite: páginas, componentes, estilos e temas
└─ scripts/	# gerador de dados sintéticos e as medições de docs/medicoes/
└─ docs/	# esta documentação, as medições e o gerador das entregas
└─ docker-compose.yml	# postgres, api e web; o Ollama no perfil assistente
└─ docker-compose.gpu.yml	# a GPU, por cima do primeiro

A estrutura prevista na Sprint 1 tinha um `nucleo/bindings/`, a ponte do C++ para o Python por extensão. Ela não existe: a ADR-012 trocou a extensão por um executável chamado por processo.

8. Segurança

Os controles estão especificados como requisitos não funcionais em 02 — Requisitos, parte II. Em resumo: senha em hash com derivação lenta, sessão em cookie `HttpOnly` com renovação do identificador no login, bloqueio após tentativas repetidas, autorização verificada no servidor em todos os endpoints, consultas parametrizadas, escape de toda saída, limite de tamanho de entrada e mensagens de erro genéricas com o detalhe apenas no log.

Cada um desses pontos tem teste automatizado, e todos rodam na CI a cada Pull Request (H17, H70):

Requisito	O vetor	O teste que o fecha
RNF10	Cookie legível por script; fixação de sessão	<code>test_cookie_de_sessao_tem_as_marcacoes_do_rnf10</code> , <code>test_novo_login_invalida_o_identificador_anterior</code>
RNF11	Adivinhar senha; descobrir quais logins existem	<code>test_bloqueia_apos_cinco_falhas</code> , <code>test_usuario_inexistente_e_senha_errada_respondem_igual</code>
RNF12	Script do usuário executado no navegador	<code>test_seguranca.py</code> (a API só devolve JSON ou CSV, com <code>nosniff</code> , e o texto volta como foi gravado); <code>web/src/seguranca.test.jsx</code> (nenhum código da tela pula o escape do React; telas renderizadas com carga de script)
RNF12	Fórmula na planilha exportada (injeção de CSV)	<code>test_a_exportacao_neutraliza_a_formula</code> (parceiros), <code>test_a_exportacao_neutraliza_formula_no_texto_da_mensagem</code> (mensagens); as duas passam por <code>app/planilha.py</code>
RNF13	Injeção SQL	<code>test_nenhuma_consulta_e_montada_com_texto</code> (lê o código de <code>app/</code>); <code>test_carga_de_injecao_em_todo_parametro_de_toda_rota_de_leitura</code> ; o login, a busca e o cadastro com carga
RNF14	Rota sem autorização no servidor	<code>test_autorizacao.py</code> : toda rota contra todo perfil; <code>test_troca_de_perfil_vale_na_requisicao_seguinte</code> , <code>test_desativar_derruba_as_sesoes_abertas</code> , <code>test_o_cookie_forjado_nao_abre_nada</code>
RNF15	Entrada grande demais	<code>test_entrada_grande_demais_e_recusada</code> , <code>test_arquivo_grande_demais_e_recusado</code>
RNF18, RNF19	Rastreamento de pilha na resposta	<code>test_falha_inesperada_responde_generico_com_correlacao</code>

O que reprova o PR que abre um vetor são os testes derivados, e não os escritos rota a rota. Rota nova entra sozinha na matriz de autorização, que reprova se ela não declarar permissão; no fuzz de injeção, que manda cada carga em cada parâmetro dela; e na leitura do código, que recusa SQL montado com texto e as formas de pular o escape do React. Nenhum deles depende de alguém lembrar de escrever o teste. Cada teste derivado foi conferido contra o defeito que procura: com o `nosniff` desligado, a neutralização do CSV desfeita ou um `dangerouslySetInnerHTML` plantado, eles reprovam.

O texto do modelo de linguagem (H60, H65) chega à tela pelo mesmo caminho, e a leitura do código da tela já o cobre. O serviço do Ollama não tem autenticação, e por isso a porta dele não é publicada no host: só a API o alcança, pela rede do Compose (ADR-013).

Parte II — Modelo de dados

Projeto: Growth Intelligence Hub (GIH) **Entrega:** Sprint 02 acadêmica — itens 3 (MER), 4 (modelo relacional) e 6 (banco criado) **Versão:** 3.0 — 28/09/2026, o banco construído

Como o esquema cresceu. A versão 1.0 tinha 14 tabelas; a 2.0, da entrega da Sprint 02, **16**: a autenticação acrescentou `sessao_acesso` e `tentativa_login`. A construção trouxe mais quatro, cada uma com a história que a pediu: `configuracao_segmentacao` (H34), `treino_modelo` (H42), `execucao_benchmark` (H57) e `lote_mensagens` (H60). São **20**. A contagem vem de consulta ao banco, e não de memória — a evidência está na seção 6.

Este documento apresenta **dois modelos**, na ordem em que a modelagem acontece:

- 1. **Modelo conceitual (MER)** — entidades, atributos e relacionamentos, na linguagem do negócio
 - 2. **Modelo relacional** — as tabelas, com tipos, chaves primárias, chaves estrangeiras e restrições
- O primeiro responde *"o que o sistema precisa saber"*. O segundo, *"como isso vira tabela"*.

1. Modelo conceitual (MER)

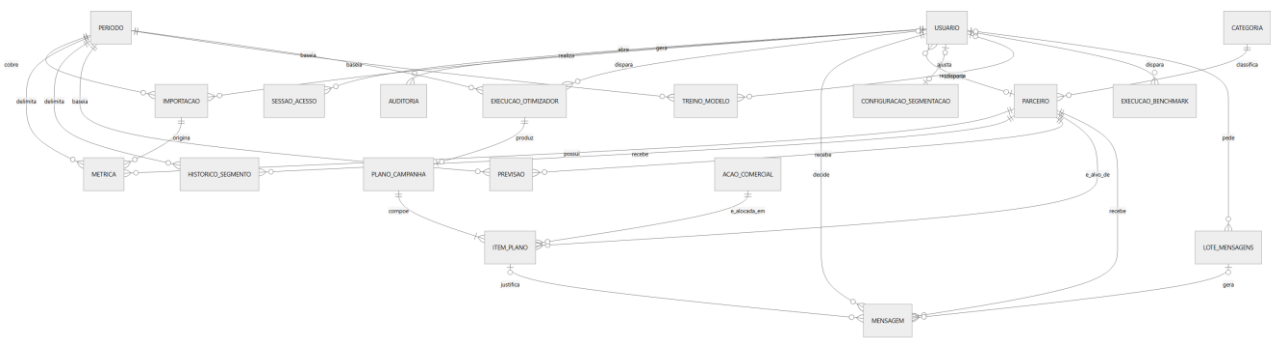


Figura 2 — Modelo entidade-relacionamento: o domínio do sistema

TENTATIVA_LOGIN não aparece no diagrama porque não se relaciona com nada. É deliberado: ela registra tentativas de autenticação inclusive contra logins que não existem, e por isso **não** pode ter chave estrangeira para **USUARIO**. Ligá-la quebraria justamente o caso que ela existe para cobrir — o ataque por dicionário usa login desconhecido (RNF11).

Impresso, o MER inteiro fica pequeno demais para ler. No repositório, o GitHub o mostra em tamanho real; no papel, cada parte da seção 2 abre com o recorte dela — o acesso, os parceiros e o desempenho, a campanha e a comunicação —, e o treino e as previsões têm o seu na 2.4.

Os atributos de cada entidade estão na tabela abaixo, e não dentro das caixas do desenho: com dezenove entidades e mais de cem atributos, a figura ficaria ilegível impressa, que é critério de aceite da entrega.

Entidades e atributos, na linguagem do negócio

Entidade	O que representa	Atributos
Usuario	Quem acessa o sistema	login, nome, senha (armazenada só como hash), perfil, situação, data de cadastro
SessaoAcesso	Uma sessão autenticada em curso	identificador da sessão, início, expiração, revogação e motivo, origem, navegador
TentativaLogin	Uma tentativa de autenticação, bem ou malsucedida	login tentado, origem, resultado, momento
Auditoria	Uma ação sensível praticada no sistema	autor, ação, parâmetros, origem, momento
Categoria	Ramo de atuação do parceiro	nome, situação
Parceiro	Comércio da rede	nome, categoria, origem da categoria, status comercial, contato, situação
Periodo	Janela de tempo de um relatório	data inicial, data final
Importacao	Uma carga de relatório	período coberto, autor, origem, total gravado, total rejeitado, momento
Metrica	Desempenho de um parceiro em um período	faturamento, número de pedidos, projeção

Entidade	O que representa	Atributos
HistoricoSegmento	Classificação de um parceiro em um período	segmento, momento do cálculo
ConfiguracaoSegmentacao	Os limiares de RN01, configuráveis (RF21)	tamanho do Top N, períodos de tendência, períodos para ser recém-chegado, quem alterou e quando
Previsao	Estimativa do modelo para um parceiro	faturamento previsto, probabilidade de queda, versão do modelo
TreinoModelo	Uma execução do treino do modelo preditivo (RF27)	autor, situação, período-base, volume de dados, métricas lado a lado com as referências, se entrou em uso, versão em uso depois dele, motivo, pesos
AcaoComercial	Tipo de ação que a campanha pode alocar	nome, custo unitário, efeito de crescimento, efeito de retenção (RN10), situação
ExecucaoOtimizador	Uma rodada do otimizador (UC08)	autor, situação, modo, parâmetros, período e versão das previsões usadas, semente, início e fim, viabilidade, restrição violada e motivo, ganho, custo, tempo, se foi parcial
PlanoCampanha	O plano resultante de uma execução viável	janela de aplicação
ItemPlano	Par (parceiro, ação) escolhido pelo otimizador	uplift esperado, custo
ExecucaoBenchmark	Uma medição do benchmark (UC09): o mesmo problema sintético em cada modo	autor, situação, parceiros, tipos de ação, repetições, semente, início e fim, andamento, as medidas cruas de cada modo, onde mediu, se outro cálculo disputou a máquina, motivo
Mensagem	Comunicação gerada para um parceiro	texto gerado, texto final, estado, autor da decisão, motivo da rejeição

Quatro relacionamentos que carregam regra de negócio

- **Metrica é única por (Parceiro, Período).** Não é detalhe técnico: é o que impede uma reimportação de duplicar a série e corromper a segmentação por tendência.
- **ExecucaoOtimizador produz zero ou um PlanoCampanha.** RN07: ou o plano respeita todas as restrições, ou não existe plano. A execução inviável fica registrada com o motivo.
- **PlanoCampanha compõe um ou mais ItemPlano,** e no máximo um por parceiro. Item sem plano não tem significado.
- **Usuario representa zero ou um Parceiro.** Só o perfil **PARCEIRO** usa esse vínculo, e é ele que restringe aquele usuário a consultar o próprio desempenho (RF26).

2. Modelo relacional

Notação: **sublinhado** é chave primária, * marca chave estrangeira.

```

usuario(id, login, nome, senha_hash, perfil, ativo, parceiro_id*, criado_em)
sessao_acesso(id, token_hash, usuario_id*, criada_em, expira_em, revogada_em, motivo_revogacao,
origem, agente)
tentativa_login(id, login, origem, sucesso, ocorrido_em)
auditoria(id, usuario_id*, acao, detalhes, origem, ocorrido_em)

categoria(id, nome, ativa)
parceiro(id, nome, nome_normalizado, categoria_id*, origem_categoria, status, contato, ativo,
criado_em)

periodo(id, data_inicio, data_fim)
importacao(id, periodo_id*, usuario_id*, origem, total_gravado, total_rejeitado, enviado_em)
metrica(id, parceiro_id*, periodo_id*, importacao_id*, faturamento, pedidos, projecao)
historico_segmento(id, parceiro_id*, periodo_id*, segmento, calculado_em)
configuracao_segmentacao(id, top_n, periodos_tendencia, periodos_novato, atualizado_em,
atualizado_por_id*)

previsao(id, parceiro_id*, periodo_base_id*, faturamento_previsto, probabilidade_queda,
modelo_versao, gerada_em)
treino_modelo(id, situacao, usuario_id*, periodo_base_id*, semente, iniciado_em, concluido_em,
parceiros, periodos, amostras_treino, amostras_validacao, amostras_teste,
mape_modelo, mape_ultimo, mape_media_movel, brier_modelo, brier_referencia,
```

```

calibracao_modelo, calibracao_referencia, detalhes, promovido, versao_em_uso,
motivo, pesos)
acao_comercial(id, nome, custo_unitario, efeito_crescimento, efeito_retencao, ativa)
execucao_otimizador(id, situacao, usuario_id*, modo, parametros, periodo_base_id*, modelo_versao,
semente, iniciada_em, concluida_em, viavel, restricao_violada, uplift_total,
custo_total, tempo_ms, parcial, motivo, detalhes)
plano_campanha(id, execucao_id*, aplicacao_inicio, aplicacao_fim)
item_plano(id, plano_id*, parceiro_id*, acao_id*, uplift_esperado, custo)
execucao_benchmark(id, situacao, usuario_id*, parceiros, acoes, repeticoes, semente, iniciada_em,
concluida_em, progresso, resultados, ambiente, disputada, motivo)

lote_mensagens(id, situacao, usuario_id*, publico, alvos, falhas, modelo, iniciado_em,
concluido_em, motivo)
mensagem(id, parceiro_id*, item_plano_id*, lote_id*, segmento, fatos, redator, modelo,
motivo_redator, texto_gerado, texto_final, estado, decidida_por_id*, motivo_rejeicao,
gerada_em, decidida_em)

```

Toda tabela tem chave primária `id` inteira e sequencial. A escolha por chave substituta, e não por chave natural composta, é o que mantém as chaves estrangeiras com uma coluna só — e o que permite corrigir um nome de parceiro sem reescrever as métricas dele.

2.1 Acesso e auditoria

O recorte do acesso: o usuário, a sessão, a trilha, o parceiro que ele representa e os limiares que ajusta.

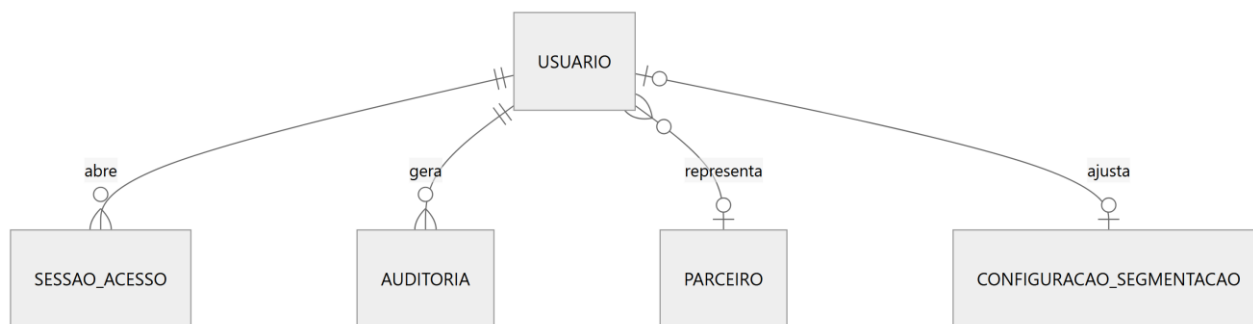


Figura 3 — Modelo entidade-relacionamento: o acesso

usuario

Coluna	Tipo	Chave	Restrição
id	serial	PK	
login	varchar(60)		único
nome	varchar(120)		
senha_hash	varchar(255)		somente hash Argon2id (RNF09)
perfil	enum		ADMINISTRADOR, GESTOR, ANALISTA, PARCEIRO
ativo	boolean		padrão verdadeiro
parceiro_id	integer	FK → parceiro	nulo, exceto no perfil PARCEIRO
criado_em	timestampz		padrão now()

CHECK (perfil = 'PARCEIRO') = (parceiro_id IS NOT NULL) — ter vínculo e ser do perfil Parceiro são a mesma condição, nos dois sentidos.

sessao_acesso

Coluna	Tipo	Chave	Restrição
id	serial	PK	
token_hash	varchar(64)		único — SHA-256 do identificador, nunca ele próprio
usuario_id	integer	FK → usuario	

Coluna	Tipo	Chave	Restrição
criada_em	timestampz		padrão now()
expira_em	timestampz		
revogada_em	timestampz		nulo enquanto a sessão vale
motivo_revogacao	varchar(40)		LOGOUT, NOVO_LOGIN, SENHA_ALTERADA, USUARIO_DESATIVADO
origem	varchar(45)		cabe IPv6
agente	varchar(255)		

tentativa_login

Coluna	Tipo	Chave	Restrição
id	serial	PK	
login	varchar(60)		texto livre, não FK — ver seção 1
origem	varchar(45)		
sucesso	boolean		
ocorrido_em	timestampz		padrão now()

auditoria

Coluna	Tipo	Chave	Restrição
id	serial	PK	
usuario_id	integer	FK → usuario	nulo quando o login tentado não existe
acao	varchar(60)		
detalhes	jsonb		nunca senha nem identificador de sessão
origem	varchar(45)		
ocorrido_em	timestampz		padrão now()

2.2 Parceiros

O recorte dos parceiros e dos dados de desempenho, que as seções 2.2 e 2.3 detalham:

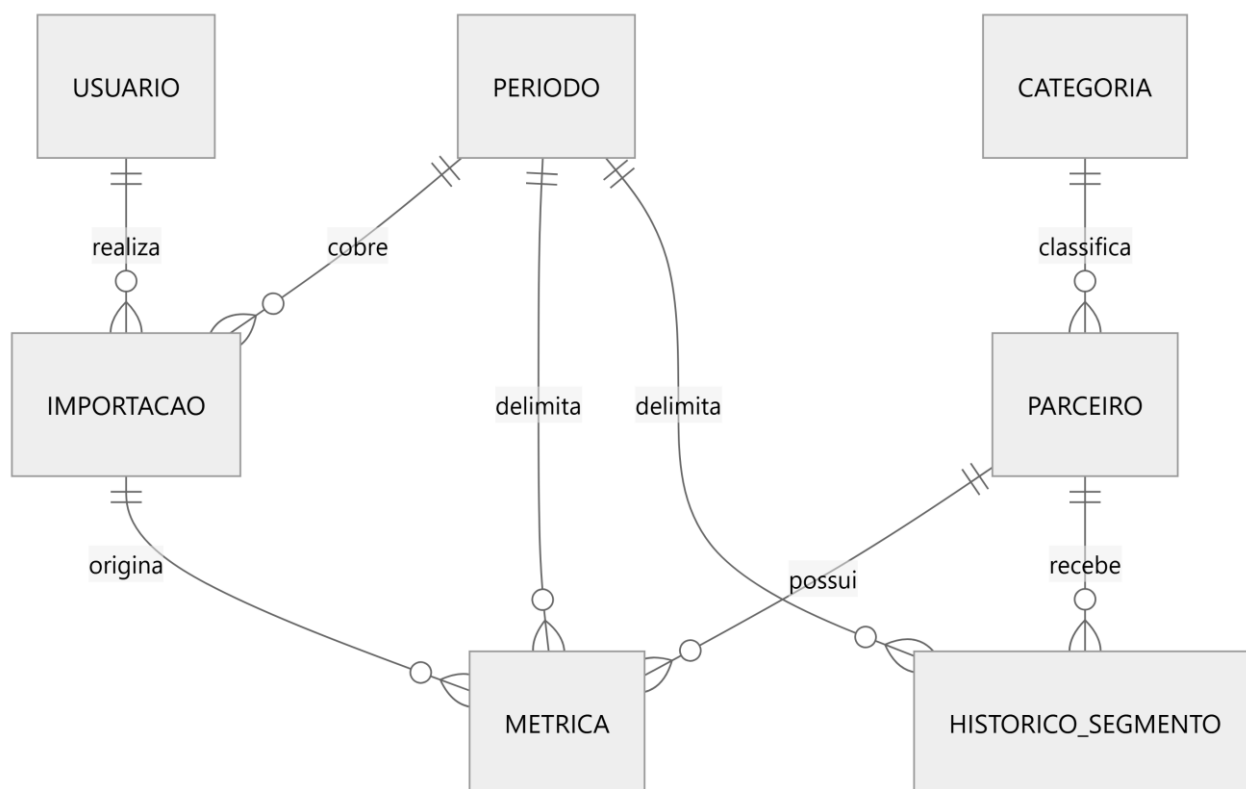


Figura 4 — Modelo entidade-relacionamento: os parceiros e os dados de desempenho

categoria

Coluna	Tipo	Chave	Restrição
id	serial	PK	
nome	varchar(60)		único
ativa	boolean		padrão verdadeiro

parceiro

Coluna	Tipo	Chave	Restrição
id	serial	PK	
nome	varchar(160)		único , indexado
nome_normalizado	varchar(160)		sem acento e sem caixa; índice de trígama (RF24)
categoria_id	integer	FK → categoria	nulo permitido
origem_categoria	enum		INFERIDA, SUGERIDA_IA, MANUAL
status	enum		ATIVO, INATIVO, PROSPECCAO
contato	varchar(120)		
ativo	boolean		padrão verdadeiro
criado_em	timestampz		padrão now()

CHECK (categoria_id IS NULL) = (origem_categoria IS NULL) — categoria sem origem seria uma classificação sem procedência, e RN05 depende de saber se ela foi confirmada por alguém.

nome_normalizado entrou na Sprint 6 (H37). unaccent(lower(nome)) na consulta resolveria o acento e impediria qualquer índice de servir, porque a função é aplicada linha a linha. A forma normalizada é coluna, gravada pela aplicação a cada alteração do nome, e o índice GIN de trígama é o único que faz LIKE '%termo%' usar índice em qualquer posição do nome.

2.3 Dados de desempenho

periodo

Coluna	Tipo	Chave	Restrição
id	serial	PK	
data_inicio	date		único em conjunto com data_fim
data_fim	date		CHECK data_fim >= data_inicio

importacao

Coluna	Tipo	Chave	Restrição
id	serial	PK	
periodo_id	integer	FK → periodo	
usuario_id	integer	FK → usuario	autor da carga (RF13)
origem	enum		TEXTO, CSV
total_gravado	integer		padrão 0
total_rejeitado	integer		padrão 0
enviado_em	timestampz		padrão now()

metrica — a tabela central do sistema

Coluna	Tipo	Chave	Restrição
id	serial	PK	
parceiro_id	integer	FK → parceiro	único em conjunto com periodo_id
periodo_id	integer	FK → periodo	
importacao_id	integer	FK → importacao	de qual carga o registro veio
faturamento	numeric(12,2)		CHECK >= 0
pedidos	integer		CHECK >= 0
projecao	numeric(12,2)		nulo permitido

Não existe coluna **ticket_medio**. Ele é faturamento dividido por pedidos, calculado na consulta (RN04). Como coluna, divergiria das parcelas que o originam na primeira correção de dado.

historico_segmento

Coluna	Tipo	Chave	Restrição
id	serial	PK	
parceiro_id	integer	FK → parceiro	único em conjunto com periodo_id
periodo_id	integer	FK → periodo	
segmento	enum		TOP, EM_ASCENSAO, EM_RISCO, RECEM_CHEGADO, ESTAVEL, PROSPECCAO
calculado_em	timestampz		padrão now()

configuracao_segmentacao — entrou na Sprint 7 (H34)

Coluna	Tipo	Chave	Restrição
id	integer	PK	CHECK id = 1 — uma linha só
top_n	integer		CHECK >= 1
periodos_tendencia	integer		CHECK >= 1

Coluna	Tipo	Chave	Restrição
periodos_novato	integer		CHECK >= 1
atualizado_em	timestampz		padrão now()
atualizado_por_id	integer	FK → usuario	nulo nos valores de fábrica

Uma linha só, garantida pelo banco. Configuração global sem essa trava vira duas linhas na primeira gravação concorrente, e a regra passa a depender de qual delas o `SELECT` devolver primeiro. A linha nasce com a migração, com os valores de RN01: Top 15, 2 períodos de tendência, 3 períodos para ser recém-chegado — este último ainda sem aval do PO (issue #59).

2.4 Núcleo computacional

previsao

Coluna	Tipo	Chave	Restrição
id	serial	PK	
parceiro_id	integer	FK → parceiro	único em conjunto com periodo_base_id e modelo-versao
periodo_base_id	integer	FK → periodo	
faturamento_previsto	numeric(12,2)		
probabilidade_queda	double		CHECK entre 0 e 1
modelo-versao	varchar(40)		versionar é o que permite comparar modelos
gerada_em	timestampz		padrão now()

Previsão e treino se ligam pela versão, não por chave estrangeira. `modelo-versao` é `rede-7` quando a previsão saiu da rede treinada no treino 7, e `referencia-7` quando saiu das contas simples medidas nele — o que vale enquanto nenhuma versão da rede superou as referências (UC07-A1, RN09). O nome carrega a origem, e é o que a tela mostra ao lado da estimativa.

treino_modelo — entrou na Sprint 05 da disciplina (H42 a H45, ADR-010)

O recorte do modelo que o módulo de previsão usa — o MER inteiro, na seção 1, fica ilegível impresso na largura de uma página:

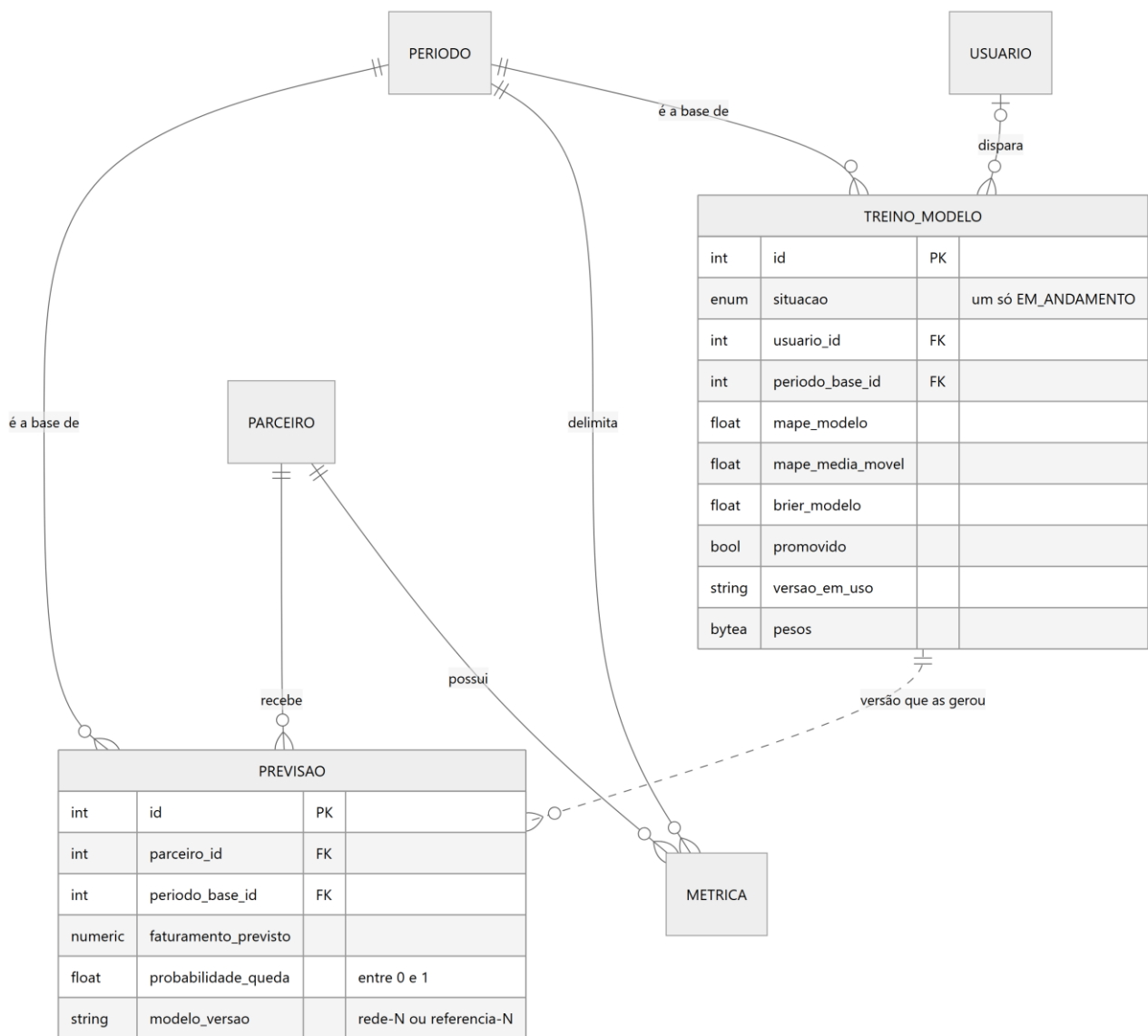


Figura 5 — Modelo entidade-relacionamento: o treino e as previsões

A ligação entre treino e previsão é **tracejada** porque não é chave estrangeira: vem da versão, pelo nome (rede-7 saiu do treino 7).

Coluna	Tipo	Chave	Restrição
id	serial	PK	
situacao	enum		EM_ANDAMENTO, CONCLUIDO, FALHOU — um só em andamento , por índice único parcial
usuario_id	integer	FK → usuario	nulo quando o treino veio do terminal
periodo_base_id	integer	FK → periodo	o período mais recente com dado — de onde as previsões partem
semente	integer		a do treino, para que ele possa ser refeito (RNF16)
iniciado_em, concluido_em	timestampz		a data do treino que o RF27 pede
parceiros, periodos, amostras_treino, amostras_validacao, amostras_teste	integer		o volume de dados (RF27)
mape_modelo, mape_ultimo, mape_media_movel	double		faturamento: a rede e as duas referências (H42, H46)
brier_modelo, brier_referencia, calibracao_modelo, calibracao_referencia	double		risco: a rede e a taxa observada (H43)

Coluna	Tipo	Chave	Restrição
detalhes	jsonb		curva de calibração, épocas, temperatura, duração, taxas da referência
promovido	boolean		a versão treinada entrou em uso (UC07-A1)
versao_em_uso	varchar(40)		a que ficou valendo depois deste treino
motivo	text		por que não entrou em uso, ou por que falhou
pesos	bytea		a rede treinada, com a normalização — poucos KB (ADR-010)

CHECK que exige versão em uso de todo treino concluído, e motivo de todo treino que falhou. **A versão em uso não fica em configuração à parte:** é o que o último treino concluído registra. Duas fontes para a mesma resposta acabariam discordando.

O recorte da campanha: a execução do otimizador, o plano, os itens, as ações e o benchmark.

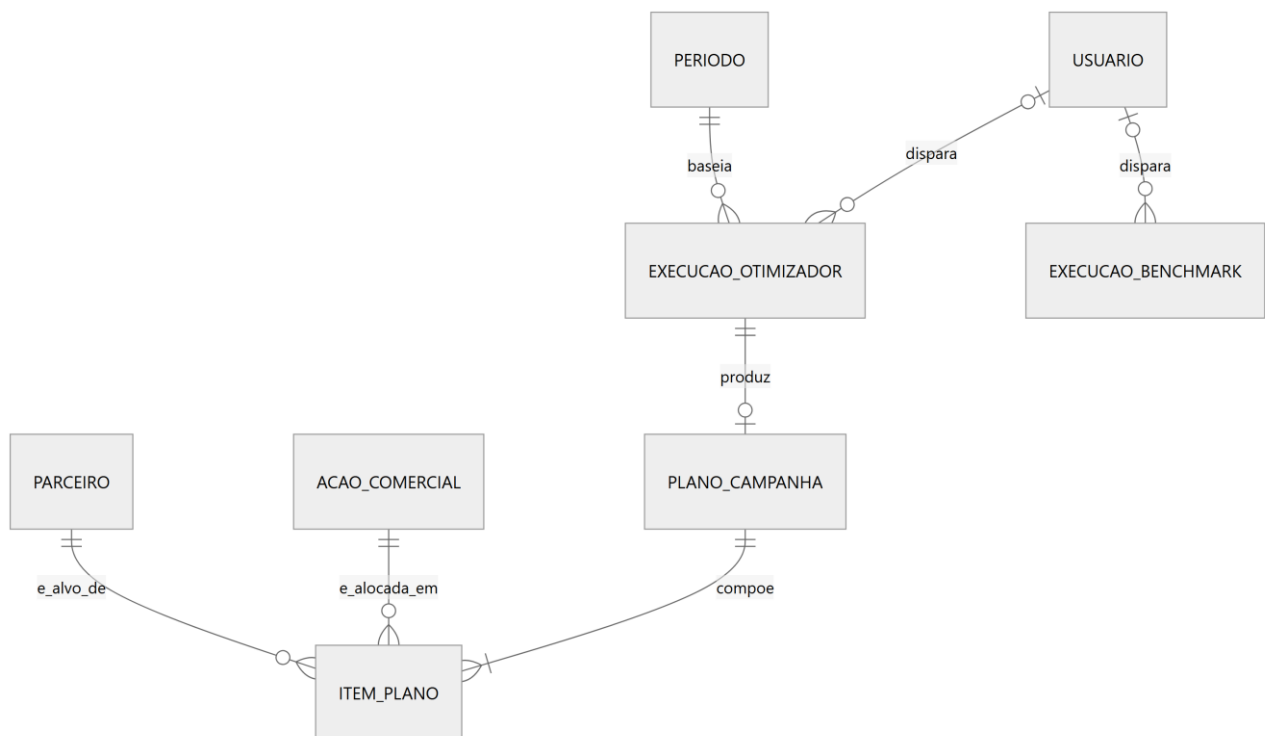


Figura 6 — Modelo entidade-relacionamento: a campanha e o benchmark

acao_comercial — os efeitos entraram na Sprint 9 interna (RN10, issue #117)

Coluna	Tipo	Chave	Restrição
id	serial	PK	
nome	varchar(80)		único
custo_unitario	numeric(10,2)		CHECK > 0 — a violação de orçamento é medida em ações da mais barata (ADR-011)
efeito_crescimento	numeric(5,4)		fração do faturamento previsto que a ação acrescenta; CHECK entre 0 e 1
efeito_retencao	numeric(5,4)		fração que ela preserva quando o parceiro cairia; CHECK entre 0 e 1
ativa	boolean		padrão verdadeiro; a inativa não entra na campanha

Os efeitos são **numeric**, e não **double**: o ganho de cada ação é calculado em **centavos inteiros**, e **float** mudaria o último centavo conforme o arredondamento binário.

execucao_otimizador — passou a registrar a busca em segundo plano na Sprint 9 interna (H49 a H52, ADR-011)

Coluna	Tipo	Chave	Restrição
id	serial	PK	
situacao	enum		EM_ANDAMENTO, CONCLUIDA, FALHOU — uma só em andamento , por índice único parcial
usuario_id	integer	FK → usuario	nulo quando a execução veio do terminal
modo	enum		SERIAL, CPU_PARALELO, GPU
parametros	jsonb		as restrições da rodada: orçamento, máximo de ações, cotas, período de aplicação
periodo_base_id	integer	FK → periodo	o período das previsões que deram o ganho
modelo-versao	varchar(40)		a versão das previsões — com a semente, o que torna o plano reproduzível
semente	integer		
iniciada_em, concluida_em	timestampz		
viavel	boolean		nulo enquanto roda
restricao_violada	varchar(120)		o código da restrição que tornou a campanha inviável
uplift_total	numeric(12,2)		nulo quando inviável
custo_total	numeric(12,2)		nulo quando inviável
tempo_ms	integer		base do benchmark (RF33, RF34); nulo enquanto roda
parcial	boolean		o limite de tempo interrompeu a busca (UC08-E2)
motivo	text		por que é inviável, ou por que falhou, em texto para a pessoa
detalhes	jsonb		elegíveis e excluídos por motivo, cotas em contagem, folgas, ganho do guloso

CHECK (viavel AND restricao_violada IS NULL) OR (NOT viavel AND restricao_violada IS NOT NULL) — é RN07 no banco: execução inviável **tem** que dizer o que violou. E mais dois, no molde do treino: execução concluída tem resultado e tempo; execução que falhou tem motivo.

plano_campanha

Coluna	Tipo	Chave	Restrição
id	serial	PK	
execucao_id	integer	FK → execucao_otimizador	único — um plano por execução
aplicacao_inicio	date		
aplicacao_fim	date		

item_plano

Coluna	Tipo	Chave	Restrição
id	serial	PK	
plano_id	integer	FK → plano_campanha	único em conjunto com parceiro_id
parceiro_id	integer	FK → parceiro	no máximo uma ação por parceiro
acao_id	integer	FK → acao_comercial	
uplift_esperado	numeric(12,2)		
custo	numeric(10,2)		

execucao_benchmark — Sprint 11 interna (H57, UC09, RF33)

Coluna	Tipo	Chave	Restrição
id	serial	PK	
situacao	enum		o mesmo <code>situacaoexecucao</code> do otimizador — um só em andamento , por índice único parcial
usuario_id	integer	FK → usuario	nulo quando o benchmark veio do terminal
parceiros	integer		entre 100 e 10.000, a faixa do gerador (RF16)
acoes	integer		entre 1 e 10
repeticoes	integer		entre 1 e 10
semente	integer		a da busca; a do problema sintético fica em <code>ambiente</code>
iniciada_em, concluida_em	timestampz		
progresso	jsonb		enquanto roda: o passo, o total e o que está medindo
resultados	jsonb		um item por modo — medido, com o tempo e o ganho de cada repetição ; indisponível ou falho, com o motivo
ambiente	jsonb		onde mediu: as threads do OpenMP, a GPU, o compilador
disputada	boolean		uma otimização ou um treino rodou junto, e os tempos podem ter saído maiores
motivo	text		por que falhou

As medidas cruas, e não só a média. A média, o desvio e os ganhos se calculam delas, na leitura; e o gráfico de escalabilidade junta execuções de tamanhos diferentes (UC09, passo 7). O problema medido é sintético (`gih_nucleo.cenario`), e por isso a tabela não aponta para parceiro nem período: o benchmark mede a máquina, e não a campanha.

2.5 Comunicação

O recorte da comunicação: o lote, a mensagem, quem a decide e o item do plano que a justifica.

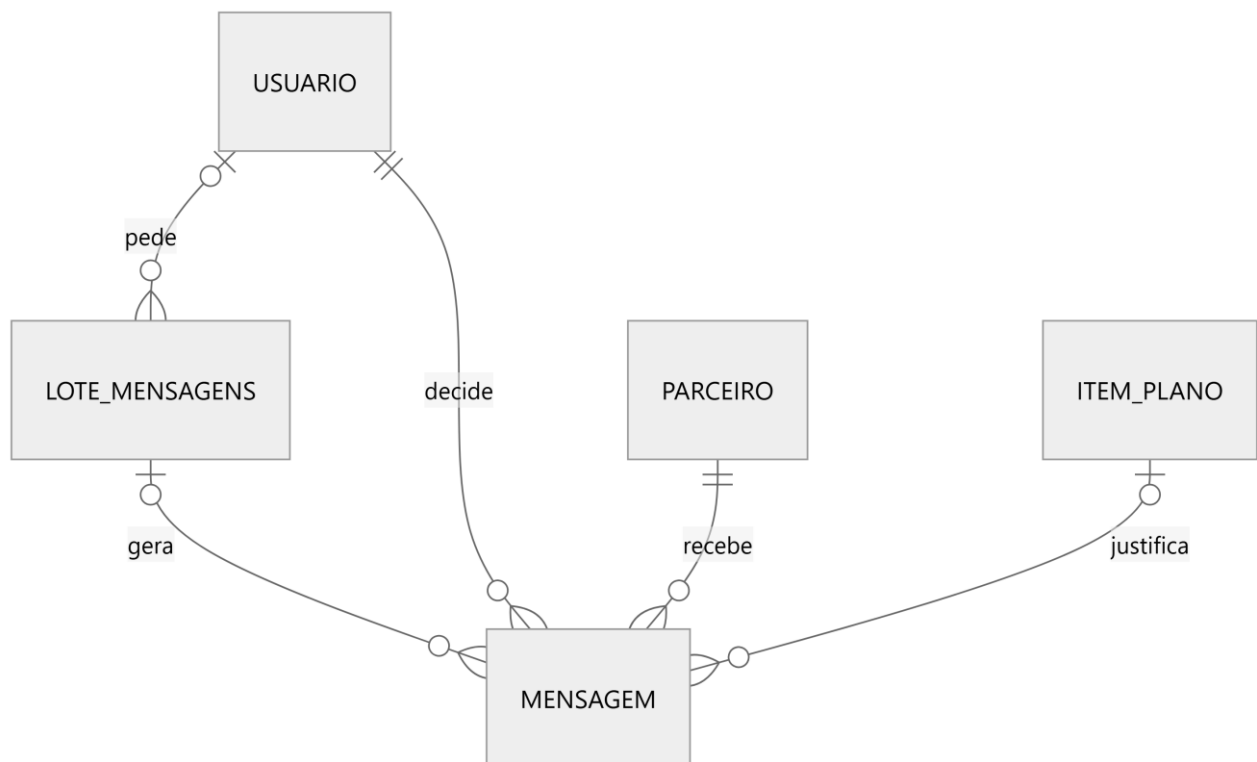


Figura 7 — Modelo entidade-relacionamento: a comunicação

lote_mensagens — Sprint 12 interna (H60, UC10, RF36; ADR-013)

Coluna	Tipo	Chave	Restrição
id	serial	PK	
situacao	enum		o mesmo <code>situacaoexecucao</code> do otimizador — um só em andamento , por índice único parcial
usuario_id	integer	FK → usuario	quem pediu
publico	jsonb		o pedido, como a pessoa o fez: segmento e categoria, plano ou seleção; e a descrição
alvos	jsonb		quem entra, resolvido no pedido : um por parceiro, com o item do plano quando há
falhas	jsonb		quem falhou, com o motivo (UC10-A2)
modelo	varchar(60)		o modelo de linguagem, se algum texto saiu dele
iniciado_em, concluido_em	timestampz		
motivo	text		por que o lote inteiro parou; obrigatório quando falhou

O público fica gravado como foi resolvido. O que a prévia mostrou é o que se gera, mesmo que um parceiro mude de segmento no meio; e o parceiro que falha fica em *falhas*, e não some (UC10-A2). O lote não aponta para o plano: os itens do plano estão em *alvos*, e cada mensagem aponta para o item dela.

mensagem

Coluna	Tipo	Chave	Restrição
id	serial	PK	
parceiro_id	integer	FK → parceiro	
item_plano_id	integer	FK → item_plano	nulo: nem toda mensagem vem de um plano
lote_id	integer	FK → lote_mensagens	a geração que a criou (H60)
segmento	enum		o do parceiro quando foi gerada: deu o tom, e a fila filtra por ele (H61)
fatos	jsonb		o que o código calculou e entregou para a redação: os números que o texto pode ter (RN08)
redator	enum		MODELO ou MODELO_FIXO (ADR-013)
modelo	varchar(60)		o modelo de linguagem, quando foi ele
motivo_redator	text		por que saiu do modelo fixo: o modelo fora do ar, ou o texto dele reprovado
texto_gerado	text		o que foi redigido — pelo modelo ou pelo modelo fixo
texto_final	text		o que o Gestor deixou; na aprovação, o texto que vale, editado ou não (RF40)
estado	enum		PENDENTE, APROVADA, REJEITADA
decidida_por_id	integer	FK → usuario	
motivo_rejeicao	varchar(240)		
gerada_em	timestampz		padrão <code>now()</code>
decidida_em	timestampz		

CHECK que impede estado diferente de **PENDENTE** sem autor e data de decisão. É **RN06 no banco**: nenhuma mensagem sai do estado pendente sem um humano registrado. Guardar `texto_gerado` e `texto_final` separados é o que permite responder depois *"o que a IA escreveu, e o que de fato foi enviado?"*.

Outro **CHECK** exige que a mensagem diga quem a redigiu: a do modelo de linguagem diz qual modelo; a do modelo fixo, por quê. Com *fatos*, isso responde à auditoria *"de onde veio cada número deste texto?"*.

3. Índices

Cada índice existe por causa de uma consulta concreta, não por precaução (RNF05).

Índice	Tabela	Colunas	Serve a
ix_parceiro_nome	parceiro	nome	Casamento de nome na importação
ix_parceiro_nome_normalizado_trgm	parceiro	nome_normalizado (GIN, trigramas)	Busca sem acento em qualquer posição do nome (RF24)
ix_metrica_periodo_faturamento	metrica	periodo_id, faturamento	Ranking do período sem varrer a tabela (RF17)
ix_segmento_periodo_segmento	historico_segmento	periodo_id, segmento	Filtro por segmento no painel (RF21)
ix_auditoria_ocorrido_em	auditoria	ocorrido_em	Consulta da trilha por intervalo (RF08)
ix_sessao_acesso_usuario	sessao_acesso	usuario_id	Derrubar as sessões de um usuário de uma vez
ix_tentativa_origem_ocorrido	tentativa_login	origem, ocorrido_em	Contagem de falhas na janela do bloqueio (RNF11)
ix_mensagem_estado	mensagem	estado	Fila de aprovação (RF37)
ix_previsao_periodo-versao	previsao	periodo_base_id, modelo-versao	As previsões de uma versão sobre um período, para a rede toda
uq_treino_um_em_andamento	treino_modelo	situacao, só onde é EM_ANDAMENTO (único, parcial)	Um treino por vez, travado pelo banco e não pela memória do processo (ADR-010)
uq_execucao_uma_em_andamento	execucao_otimizador	situacao, só onde é EM_ANDAMENTO (único, parcial)	Um otimizador por vez, pelo mesmo motivo (ADR-011)
uq_benchmark_um_em_andamento	execucao_benchmark	situacao, só onde é EM_ANDAMENTO (único, parcial)	Um benchmark por vez: dois dividiriam a máquina e mediriam a disputa (H57)
uq_lote_mensagens_um_em_andamento	lote_mensagens	situacao, só onde é EM_ANDAMENTO (único, parcial)	Uma geração de mensagens por vez (H60)
ix_mensagem_lote	mensagem	lote_id	As mensagens de um lote, que a tela consulta enquanto ele gera (UC10-A1)

4. Tipos enumerados

Dez tipos ENUM do PostgreSQL, em vez de texto livre. O banco recusa valor fora da lista, o que é mais forte que uma validação de aplicação que alguém pode esquecer de chamar.

Tipo	Valores
perfil	ADMINISTRADOR, GESTOR, ANALISTA, PARCEIRO
segmento	TOP, EM_ASCENSAO, EM_RISCO, RECEM_CHEGADO, ESTAVEL, PROSPECCAO
origemcategoria	INFERIDA, SUGERIDA_IA, MANUAL
statuscomercial	ATIVO, INATIVO, PROSPECCAO
origemimportacao	TEXTO, CSV
modoexecucao	SERIAL, CPU_PARALELO, GPU
estadomensagem	PENDENTE, APROVADA, REJEITADA
situacaotreino	EM_ANDAMENTO, CONCLUIDO, FALHOU
situacaoexecucao	EM_ANDAMENTO, CONCLUIDA, FALHOU
redatormensagem	MODELO, MODELO_FIXO

Armadilha registrada: o autogenerate do Alembic **não** remove tipos ENUM no downgrade — só derruba as tabelas. Sem acrescentar DROP TYPE à mão, reverter e reaplicar falha com *type already exists*, e o erro só aparece na segunda execução. Toda migração que cria enum precisa derrubá-lo.

5. O que o banco garante sozinho

Vinte e uma restrições CHECK que impedem estado inválido independentemente do código da aplicação. É a diferença entre uma regra que vale e uma regra que valeria se ninguém esquecesse de chamá-la.

Restrição	Garante
ck_usuario_parceiro_apenas_perfil_parceiro	Vínculo com parceiro se e somente se o perfil for PARCEIRO
ck_parceiro_categoria_com_origem	Categoria sempre acompanhada da procedência (RN05)
ck_periodo_ordem	Data final nunca anterior à inicial
ck_metrica_faturamento_nao_negativo	Faturamento ≥ 0
ck_metrica_pedidos_nao_negativo	Pedidos ≥ 0
ck_previsao_probabilidade	Probabilidade entre 0 e 1
ck_acao_custo_positivo	Custo de ação > 0 — a violação de orçamento é medida em ações da mais barata (ADR-011)
ck_acao_efeitos_entre_0_e_1	Os dois efeitos da ação são frações (RN10)
ck_execucao_inviavel_tem_motivo	Execução inviável tem motivo registrado (RN07)
ck_execucao_concluida_tem_resultado	Execução concluída diz se é viável, e quanto tempo levou
ck_execucao_falha_tem_motivo	Execução que falhou diz por quê
ck_mensagem_decisao_tem_autor	Mensagem decidida tem autor e data (RN06)
ck_treino_concluido_tem-versao	Treino concluído diz qual versão ficou em uso (UC07-A1)
ck_treino_falho_tem_motivo	Treino que falhou diz por quê
ck_benchmark_parceiros, ck_benchmark_acoes, ck_benchmark_repeticoes	O cenário do benchmark fica na faixa do gerador e da tela (UC09)
ck_benchmark_concluido_tem_resultado	Benchmark concluído tem as medidas
ck_benchmark_falha_tem_motivo	Benchmark que falhou diz por quê
ck_lote_mensagens_falha_tem_motivo	Geração de mensagens que parou diz por quê
ck_mensagem_redator_explicado	Mensagem diz quem a redigiu: o modelo, com o nome dele, ou o modelo fixo, com o motivo (ADR-013)

6. O banco criado — evidência

O esquema não está só desenhado: está aplicado e em uso. O ambiente sobe com um comando (`docker compose up`), e o entrypoint aplica as migrações antes de servir a API.

Migração aplicada:

```
$ docker compose exec api alembic current
3e8a6c1f7b52 (head)
```

Tabelas criadas (`docker compose exec postgres psql -U gih -d gih -c "\dt"`):

Schema	Name	Type	Owner
public	acao_comercial	table	gih
public	alembic_version	table	gih
public	auditoria	table	gih
public	categoria	table	gih
public	configuracao_segmentacao	table	gih
public	execucao_benchmark	table	gih
public	execucao_otimizador	table	gih
public	historico_segmento	table	gih

```

public | importacao | table | gih
public | item_plano | table | gih
public | lote_mensagens | table | gih
public | mensagem | table | gih
public | metrica | table | gih
public | parceiro | table | gih
public | periodo | table | gih
public | plano_campanha | table | gih
public | previsao | table | gih
public | sessao_acesso | table | gih
public | tentativa_login | table | gih
public | treino_modelo | table | gih
public | usuario | table | gih
(21 rows)

```

São as 20 tabelas de domínio mais `alembic_version`, que é da própria ferramenta de migração e registra qual versão do esquema está aplicada.

Contagem por consulta ao catálogo do PostgreSQL:

Objeto	Quantidade
Tabelas de domínio	20
Chaves primárias	20
Chaves estrangeiras	28
Restrições <code>UNIQUE</code>	11
Restrições <code>CHECK</code> declaradas	25
Índices	45
Tipos <code>ENUM</code>	10

Medido em 27/09/2026, contra o banco no ar, sem a tabela do Alembic. Desde a medição de 26/09 (18, 18, 25, 11, 18, 40 e 9) entraram o benchmark (H57: +1 tabela, +1 chave estrangeira, +5 `CHECK`, +2 índices) e o lote de mensagens (H60: +1 tabela, +2 chaves estrangeiras, +2 `CHECK`, +3 índices, +1 enum). A contagem de `CHECK` do catálogo inclui as quatro da configuração da segmentação, que a seção 5 não repete. **Na Sprint 02 eram 16, 16, 21, 11, 9, 34 e 7**, e a diferença até 26/09 é de quatro migrações: o nome normalizado do parceiro com o índice de trígama (Sprint 6, +1 índice), a configuração da segmentação (Sprint 7, +1 tabela, +1 chave estrangeira, +4 `CHECK`, +1 índice), o treino do modelo (Sprint 05 da disciplina, +1 tabela, +2 chaves estrangeiras, +2 `CHECK`, +3 índices contando o da chave primária, +1 enum) e a campanha (Sprint 9 interna: +1 chave estrangeira, o período-base da execução; +3 `CHECK`, contando o custo positivo que substituiu o não negativo; +1 índice; +1 enum). Os índices únicos parciais do treino e da execução não aparecem entre as restrições `UNIQUE`: são índices, e não restrições — o PostgreSQL só aceita condição (`WHERE`) em índice.

O esquema é gerado por migração versionada, não por script solto. Isso é o que permite qualquer integrante chegar ao mesmo estado a partir de um clone limpo, e é o que torna a evolução do banco auditável no histórico do repositório.

7. Verificação

O ciclo de reverter e reaplicar foi testado **duas vezes**, e não uma: a falha de `ENUM` da seção 4 só aparece na segunda execução. A migração do treino do modelo, que cria o enum `situacaotrein`, passou pelo mesmo ciclo em 24/09/2026, num banco descartável.

A da campanha, que cria o enum `situacaoexecucao`, passou em 26/09/2026 — e com uma execução do otimizador gravada antes de cada reversão, **sem autor e falha**, que o esquema anterior não representa. O downgrade apaga essas execuções antes de exigir de novo autor, resultado e tempo; sem isso, o `resetar_banco.py`, que reverte tudo até a base, quebraria no primeiro banco que tivesse uma otimização interrompida.

A do lote de mensagens, que cria o enum `redatormensagem`, passou pelo ciclo duas vezes em 27/09/2026, no banco de teste.

```

alembic upgrade head # aplica
alembic downgrade base # reverte tudo
alembic upgrade head # reaplica — é aqui que o erro apareceria

```

Além disso, a suíte de testes da API cria um banco `gih_teste` separado e aplica **as mesmas migrações** nele, em vez de montar o esquema com `create_all`. Custa alguns segundos a mais por execução e cobre a divergência entre o modelo em código e a migração — que é o defeito que ninguém percebe até a hora de implantar.

8. Onde está a implementação

Preciso de...	Está em
As entidades em código	<code>api/app/modelos.py</code>
A migração que cria o esquema	<code>api/migrations/versions/</code>
Diagrama de classes, incluindo serviços e núcleo	<code>10-diagrama-de-classes.md</code>
Decisões de arquitetura (ADR-001 a ADR-013)	<code>07-arquitetura-preliminar.md</code>
Massa de demonstração sintética	<code>scripts/gerar_dados_sinteticos.py</code>

Parte III — Classes e serviços

Projeto: Growth Intelligence Hub (GIH) **Entrega:** Sprint 02 acadêmica — arquitetura e modelagem **Versão:** 2.0 — 28/09/2026, o sistema construído (1.0 — 16/09/2026, a entrega da Sprint 02)

Por que nove diagramas e não um. O sistema tem três camadas com naturezas diferentes: entidades de domínio (dados), serviços (regra de negócio) e núcleo computacional (C++/CUDA). Espremer as três num desenho só produziria uma figura de oitenta caixas, ilegível impressa — que é exatamente o critério de aceite da entrega. Cada diagrama abaixo se lê sozinho, e o **primeiro** mostra como eles se ligam.

1. Visão de integração

Onde cada camada vive e o que atravessa a fronteira entre elas. É esta figura que responde ao item 1 da entrega — "a arquitetura deverá contemplar a integração entre o software e o componente computacional avançado".

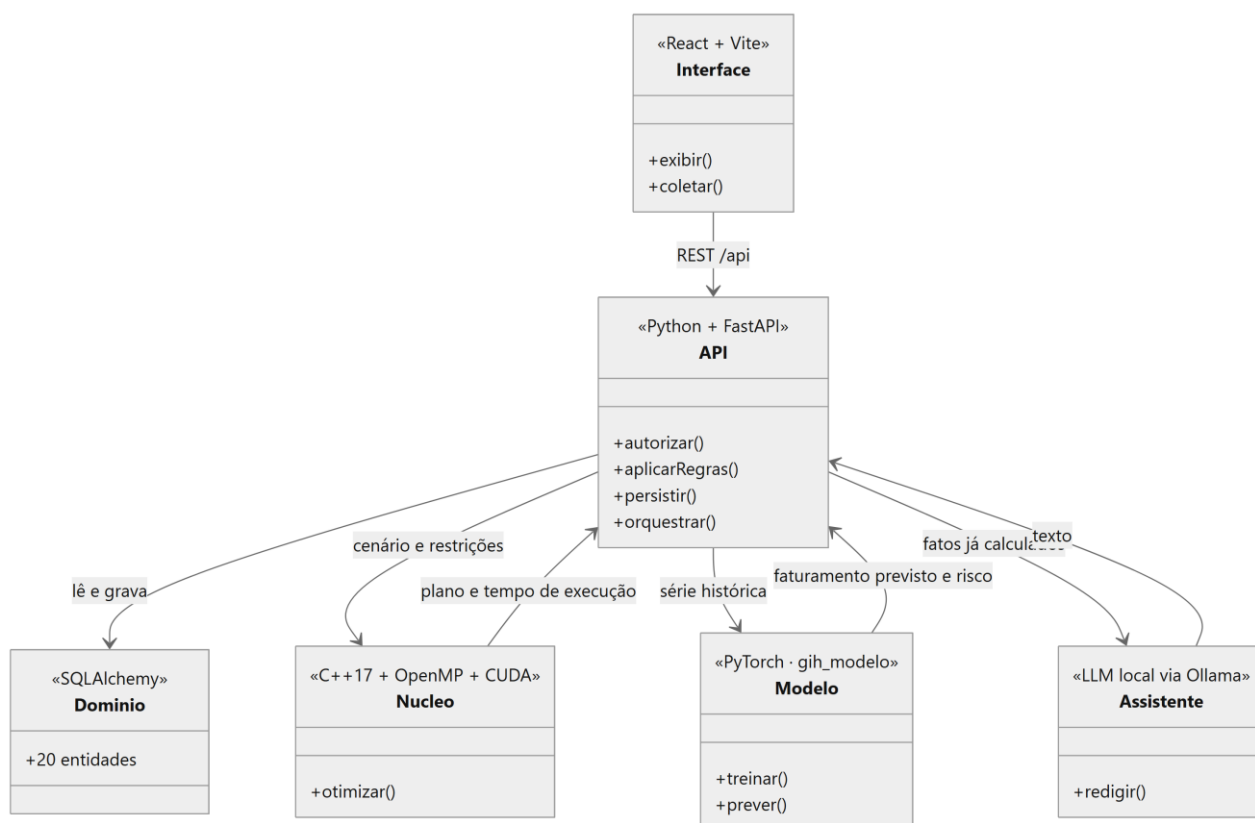


Figura 8 — Classes: a integração entre as camadas

Três fronteiras que não se atravessam, e o motivo de cada uma:

- **A interface não conhece regra de negócio.** Ela exibe e coleta; a API decide. Um `if` sobre valor de negócio no React está no lugar errado.
- **O núcleo não conhece autenticação nem banco.** Recebe um cenário e devolve um plano. Isso é o que permite compilá-lo e medi-lo fora da aplicação — foi assim que o spike da GPU rodou oito semanas antes do resto existir.
- **O assistente não calcula número.** Recebe fatos já apurados e redige texto em volta deles (RN08). Se ele somasse, o número deixaria de ser reproduzível, e a reproduzibilidade é critério de avaliação.

2. Domínio — acesso, sessões e auditoria

Quem entra, por quanto tempo, e o rastro que fica.

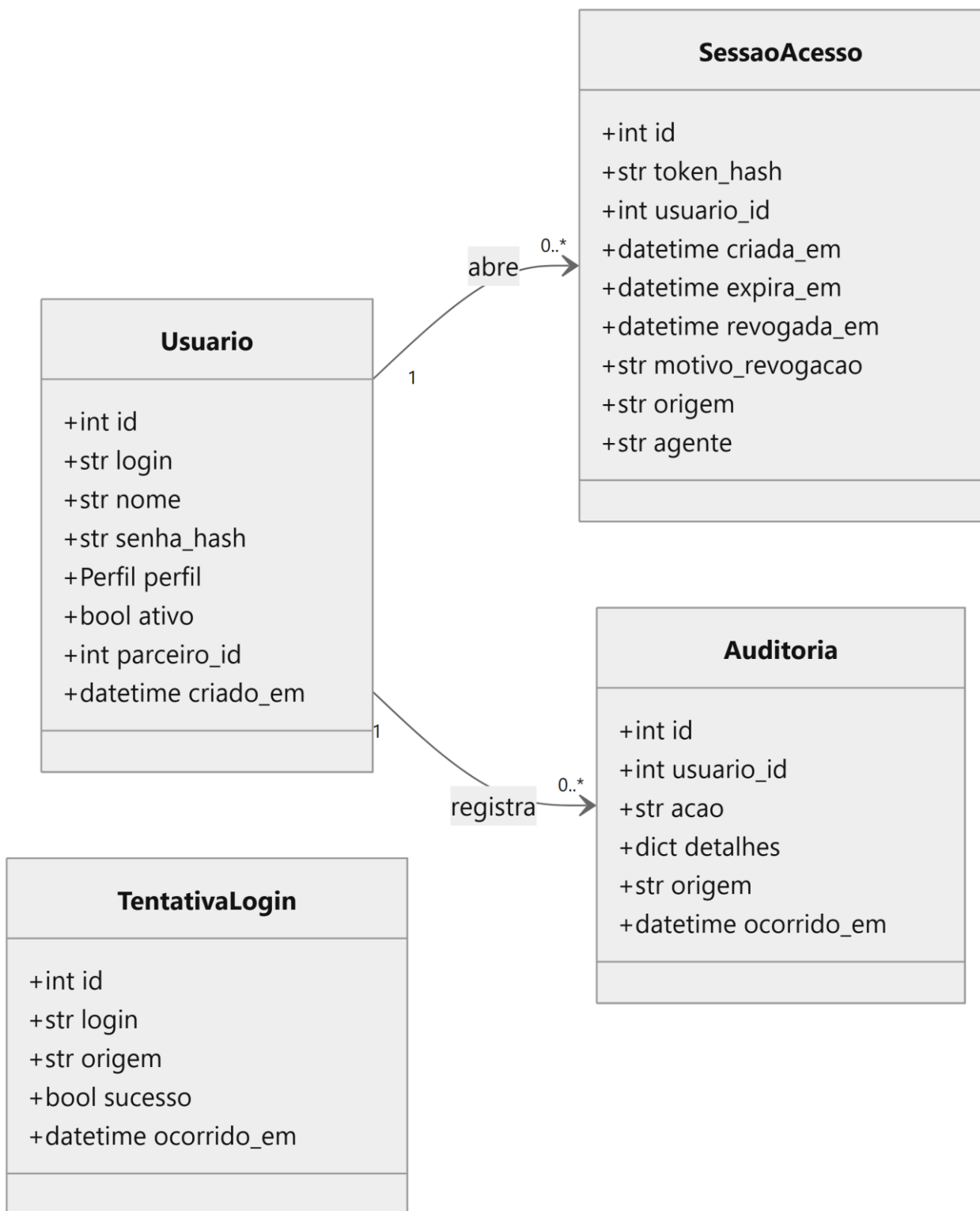


Figura 9 — Classes do domínio: acesso, usuários e auditoria

SessaoAcesso guarda o hash do identificador, nunca ele próprio. Quem conseguir ler esta tabela não consegue se passar por ninguém. E a sessão é **revogada**, não apagada: "esta sessão caiu por troca de senha" é informação de investigação, e some se a linha for removida.

TentativaLogin não se liga a Usuario, e isso é deliberado. O login é texto livre, não chave estrangeira: tentativa contra usuário inexistente também precisa ser contada, senão a defesa contra força bruta só protege quem já existe — e é justamente o login desconhecido que o ataque por dicionário usa (RNF11).

Auditoria.usuario_id é opcional pelo mesmo motivo: a tentativa de login com usuário inexistente precisa ser registrada, e nesse caso não há usuário para apontar.

3. Domínio — parceiros e dados de desempenho

O núcleo informacional do produto: quem são os parceiros, o que faturaram em cada período, e como foram classificados.

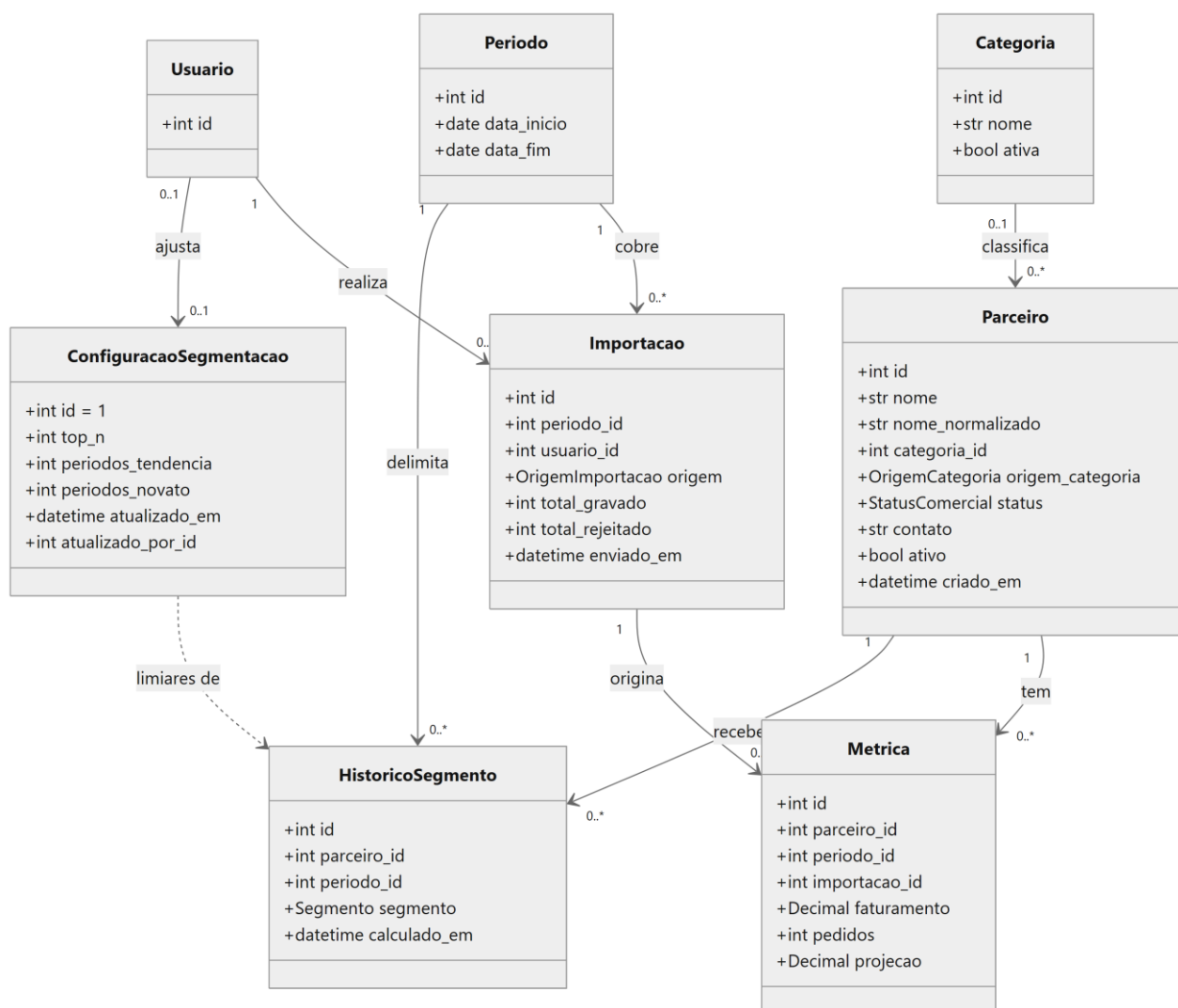


Figura 10 — Classes do domínio: parceiros, períodos e desempenho

Usuario aparece reduzido ao identificador: o detalhe dele está no diagrama anterior.

Há um vínculo a mais, omitido do desenho para não cruzá-lo de ponta a ponta: `Usuario.parceiro_id` aponta para `Parceiro`, e **só o perfil PARCEIRO o usa** — é o que restringe aquele usuário a consultar o próprio desempenho (RF26). O banco cobra isso por CHECK: ter vínculo e ser do perfil Parceiro são a mesma condição.

Metrica não tem ticket_medio. Ele é faturamento dividido por pedidos, calculado na consulta (RN04). Como atributo, divergiria das parcelas que o originam na primeira correção de dado.

Metrica é única por (parceiro, período). É essa restrição que impede uma reimportação de duplicar a série e corromper a segmentação por tendência.

Período é obrigatório e não tem valor padrão. O relatório de origem não carrega datas; sem o período informado, as métricas ficam órfãs na linha do tempo e a segmentação classifica errado **sem emitir erro** (RN03). Já aconteceu num projeto anterior do mesmo domínio.

ConfiguracaoSegmentacao entrou na Sprint 7 (H34, RF21): os três limiares de RN01 saíram do código e passaram a mudar sem alteração de código. **Uma instância só** — o banco cobra `id = 1` —, e a seta tracejada até `HistoricoSegmento` é dependência, não associação: os limiares decidem o segmento que é gravado, mas nenhum histórico aponta para a configuração que o produziu.

Parceiro.nome_normalizado entrou na Sprint 6 (H37): o nome sem acento e sem caixa, gravado pela aplicação, que é o que permite a busca sem acento usar índice.

HistoricoSegmento guarda o segmento já resolvido pela precedência de RN01. Por isso a mobilidade do Top N não pode ser derivada dele: um parceiro entre os N maiores mas em queda fica gravado como EM_RISCO, e lê-lo daqui faria o painel anunciar que ele saiu do Top N enquanto continua lá (RN02).

4. Domínio — núcleo computacional e comunicação

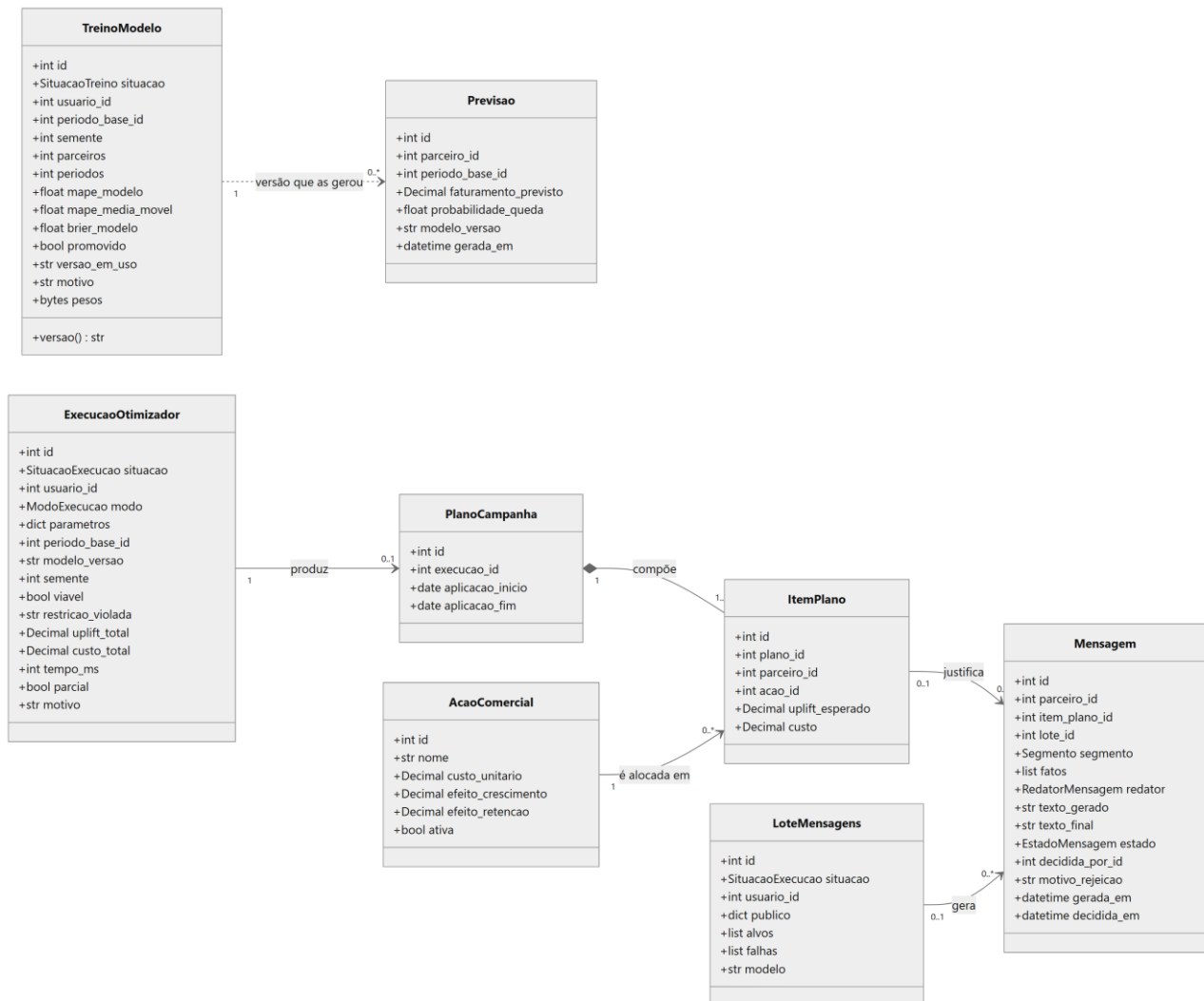


Figura 11 — Classes do domínio: campanha, otimização e comunicação

ExecucaoOtimizador existe mesmo quando não há plano. A cardinalidade 0..1 é RN07: ou o plano respeita **todas** as restrições, ou não existe plano. A execução inviável fica registrada com **restricao_violada** preenchido, porque explicar por que não deu é mais útil que sumir com a tentativa. Ela guarda também a versão do modelo, o período das previsões e a semente: com os três e os mesmos parâmetros, sai o mesmo plano (ADR-011).

AcaoComercial tem os dois efeitos da RN10. O ganho de aplicá-la a um parceiro é o que ela acrescenta ao faturamento previsto mais a parte da perda que evita quando ele cairia — por isso uma ação de retenção vale mais para quem está em risco.

PlanoCampanha **compõe** **ItemPlano** (losango cheio). Item sem plano não tem significado; apagar o plano apaga os itens.

TreinoModelo **liga-se** à **Previsao** pela **versão**, e **não por chave** (seta tracejada). **rede-7** é a rede do treino 7; **referencia-7**, as contas simples medidas nele, que valem enquanto nenhuma rede superou as referências (UC07-A1). O treino guarda a métrica da rede ao lado da de cada referência — o atributo listado é uma amostra; o modelo de dados tem todas.

Mensagem separa texto_gerado de texto_final. Se o Gestor editar antes de aprovar, o original permanece — é o que permite responder depois *"o que a IA escreveu, e o que de fato foi enviado?"*, e medir se o modelo está acertando o tom. Ela guarda também os **fatos** que o código entregou para a redação e quem a redigiu — o modelo de linguagem ou o modelo fixo, com o motivo (ADR-013).

LoteMensagens grava o público resolvido. Quem entra fica em **alvos** no momento do pedido: o que a prévia mostrou é o que se gera. Quem falha fica em **falhas**, com o motivo, e não some (UC10-A2).

5. Camada de serviços — acesso e segurança

O domínio das seções 2 a 4 é **anêmico de propósito**: as entidades carregam dados e restrições, e a regra de negócio vive nos serviços. A razão é testabilidade — a regra fica exercitável sem instanciar entidade, e o interpretador do relatório, por exemplo, é função pura de texto para resultado.

Todos os módulos estão marcados **implementado**: existem em `api/app/` e estão cobertos por teste.

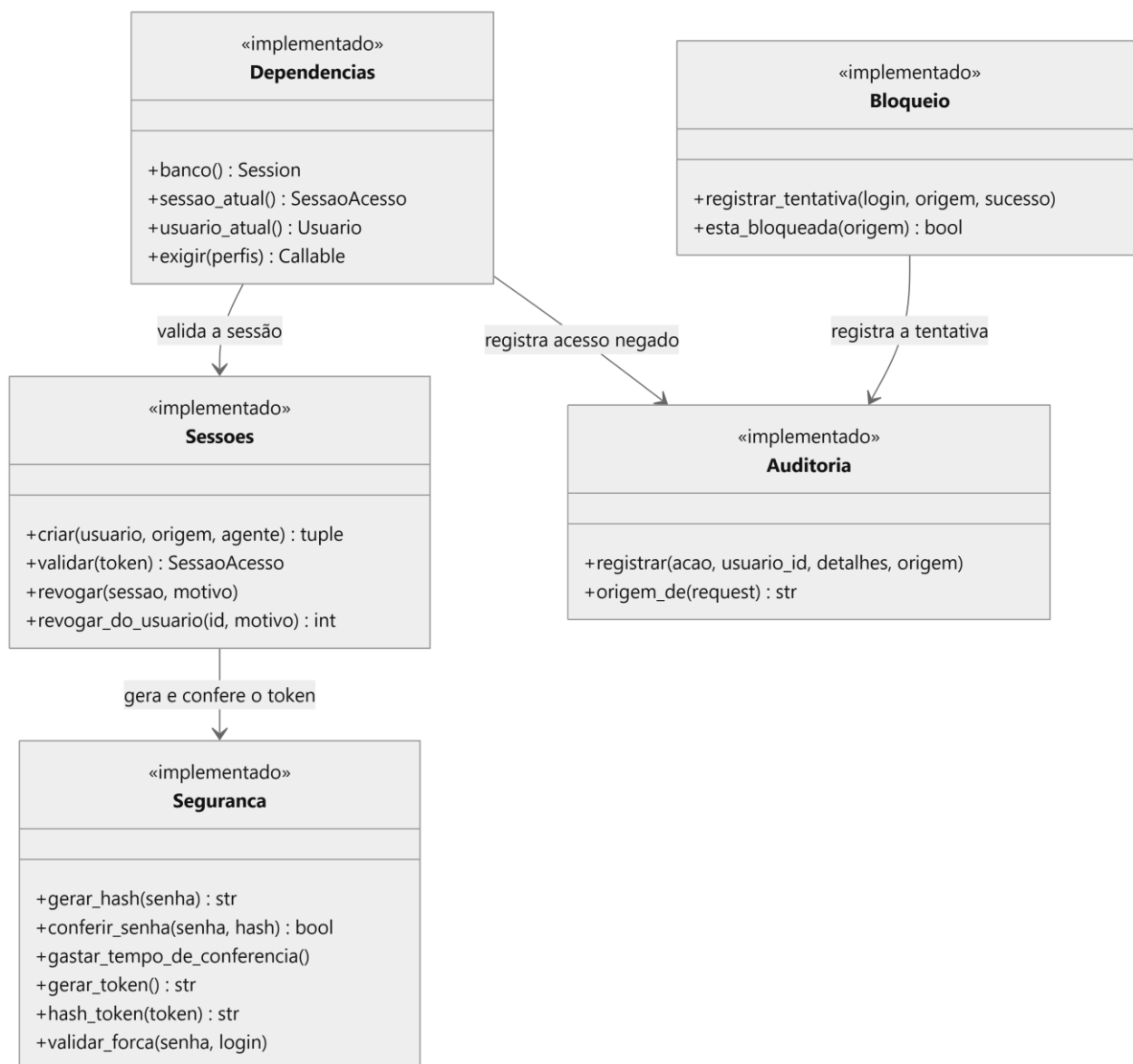


Figura 12 — Serviços: autenticação, sessão e autorização

Duas decisões desta camada que valem registro:

- **A autorização é dependência declarada na rota, não if no corpo da função.** Esquecer um `if` é silencioso; esquecer a dependência reprova o teste que percorre cada endpoint contra cada perfil (RNF14).

- **Auditoria e Bloqueio gravam em transação própria.** Se participassem da transação da requisição, o rollback de uma falha de login levaria junto o registro da falha — a trilha teria só sucessos, e o contador de força bruta apagaria as próprias evidências antes de chegar às cinco tentativas (ADR-008).

6. Camada de serviços — ingestão, análise e comunicação

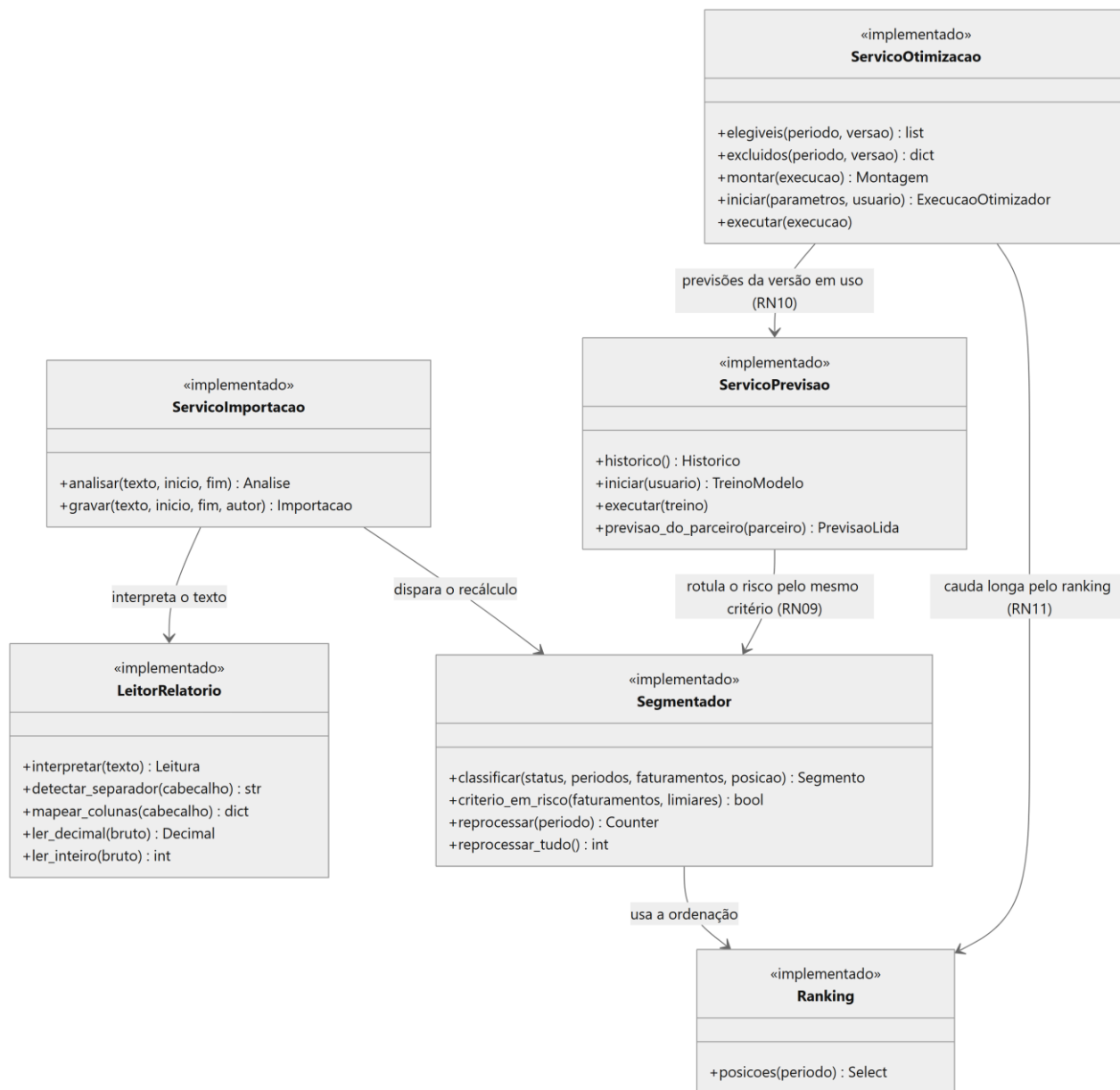


Figura 13 — Serviços: a ingestão, a análise, a previsão e a otimização

LeitorRelatorio não toca no banco. É função pura de texto para resultado, e é isso que permite a prévia da importação usar exatamente o mesmo código da gravação — sem risco de a prévia mostrar uma coisa e a gravação fazer outra.

ServicoPrevisao rotula o treino com o critério do Segmentador. A queda que o modelo aprende a prever é entrar em Em Risco no período seguinte (RN09), e o rótulo sai de `critério_em_risco` — a mesma função que classifica o segmento. O modelo em si mora fora da API, no pacote `gih_modelo` (ADR-010): recebe séries e devolve números, e quem decide se uma versão entra em uso é este serviço.

ServicoOtimizacao traduz a campanha para o núcleo, e o núcleo não sabe o que é campanha. Aqui ficam a regra de negócio e o texto: o ganho da RN10 em centavos, quem é elegível, as cotas em contagem, a cauda longa pelo ranking (a leitura da RN02) e a recusa com o nome da categoria e o valor em reais. O pacote `gih_nucleo` recebe inteiros e devolve o plano (ADR-011).

A comunicação e o assistente

Os serviços que falam com o modelo de linguagem, e a aprovação das mensagens. O Ranking e o ServicoPrevisao entram só com o nome: estão no diagrama anterior.

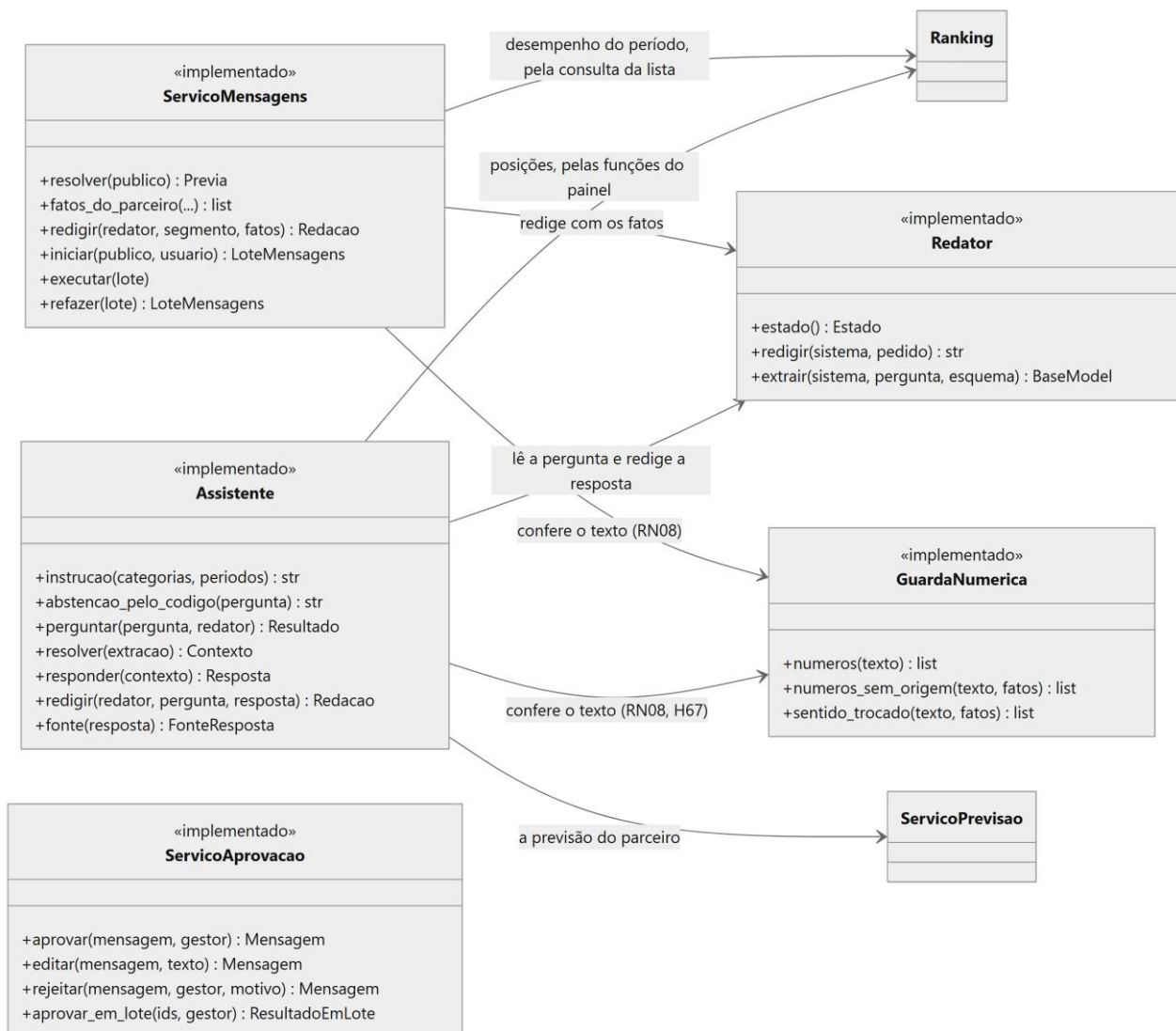


Figura 14 — Serviços: as mensagens, a aprovação e o assistente

O **Assistente** responde com as funções das telas, e o modelo só lê a pergunta (ADR-013, H65). O **Redator** devolve o tipo da pergunta, de um catálogo fechado, e os campos dela; a resolução confere os campos contra a base — o parceiro pela busca da lista de parceiros —, e cada tipo responde com a função que a tela usa: o ranking, a mobilidade, os indicadores e a previsão. Cada resposta traz os fatos que a sustentam e a fonte, que o código monta e o modelo não vê (H66). O modelo reescreve a resposta em prosa, e a **GuardaNumerica** confere o texto dele: número fora dos fatos ou variação no sentido contrário, e fica a resposta do código (H67). A pergunta que pede conta, causa ou dado que o relatório não traz recebe a abstenção antes de chegar ao modelo (H68). Se o modelo somasse, contasse ou comparasse, o número deixaria de ser reproduzível (RN08).

O **Redator** não sabe o que é parceiro, e a **GuardaNumerica** não sabe o que é modelo (ADR-013). O **Redator** fala com o Ollama: devolve texto para um pedido, ou os campos de uma pergunta num esquema, e diz por que está fora do ar quando está. A **GuardaNumerica** recebe o texto pronto e os fatos, e aponta os números do texto que os fatos não sustentam. Quem monta os fatos, e quem troca o texto reprovado pelo modelo fixo, é o serviço que usa os dois.

ServicoMensagens monta os fatos, e o segmento fica de fora deles. O desempenho do parceiro sai da mesma consulta da lista de parceiros (app/desempenho.py), para a mensagem não dizer um número que a tela não mostra. O segmento escolhe o tom e não entra no texto: a classificação é da rede, e não do parceiro (RF26). O texto do modelo passa pela guarda numérica e por uma lista de termos internos; reprovado, ou com o modelo fora do ar, sai o modelo fixo da equipe.

7. Núcleo computacional — as estruturas

A parte avaliada pela disciplina de **Tópicos Avançados**. O otimizador resolve um problema combinatório: dados N parceiros e A ações possíveis, escolher no máximo uma ação por parceiro maximizando o ganho esperado sem violar orçamento, máximo de ações e cotas. O espaço de busca é $(A+1)^N$. A formulação está na [docs/07 §4.1](#), e a versão serial, no pacote `nucleo/gih_nucleo` (Sprint 9 interna).

O que entra, o que sai, e quem avalia um candidato:

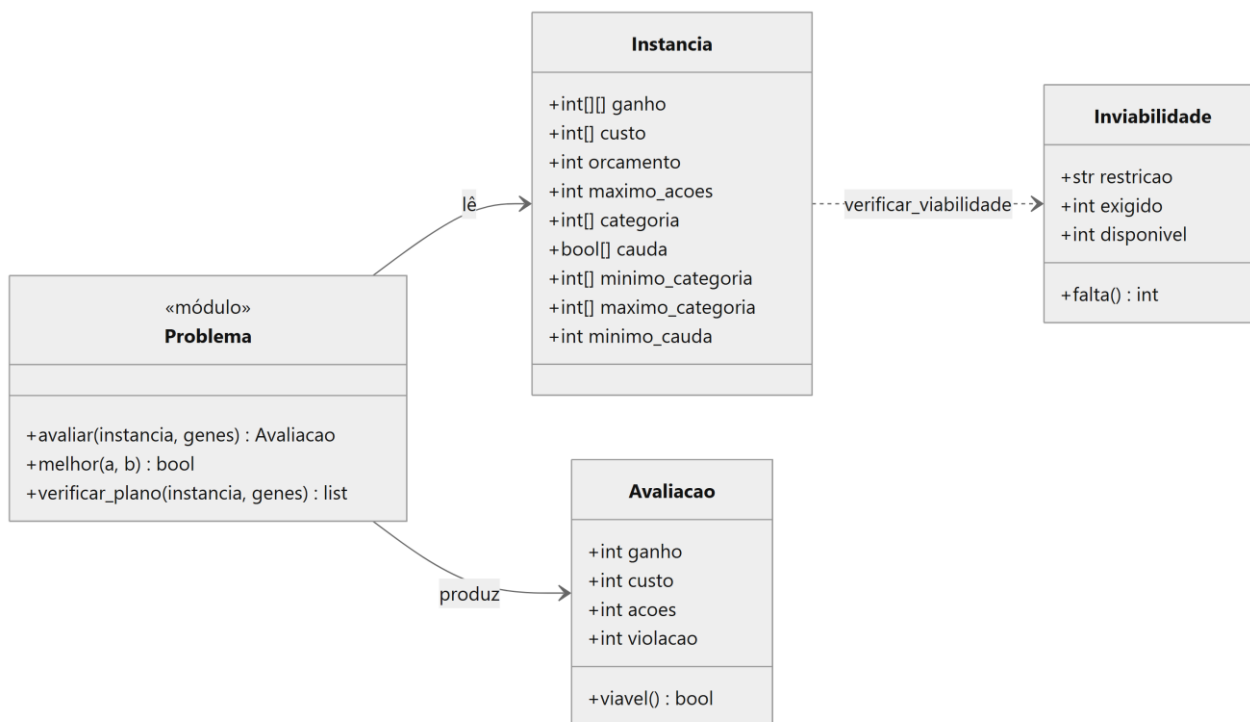


Figura 15 — Núcleo computacional: as estruturas da instância

Nenhuma dessas estruturas conhece banco, sessão ou HTTP. O núcleo recebe uma instância já montada, em **inteiros** — ganho e custo em centavos, cotas em contagem — e devolve o plano. Foi o que permitiu medir o kernel oito semanas antes de o resto do sistema existir, e é o que permite testá-lo sem subir a aplicação.

A violação é um número, e não um "inviável" que zera a aptidão. Um plano inviável compete pela violação — em unidades de ação — contra outro inviável, e perde para qualquer viável (`melhor`, ADR-011). Assim a busca pode atravessar o inviável para chegar ao ótimo; e o plano devolvido ainda passa por `verificar_plano`, escrito sem reaproveitar `avaliar` (RN07).

Inviabilidade é decidida antes da busca, com exatidão. Com as cotas em contagem, basta ver se o menor conjunto que as cumpre cabe no máximo de ações e no orçamento pagando a ação mais barata ([docs/07 §4.1](#)).

8. Núcleo computacional — as três implementações

Três implementações do **mesmo** algoritmo, para que o benchmark compare o que é comparável (RF32 a RF34). Todas recebem a **Instancia** e a semente e devolvem o mesmo plano — o sorteio é por coordenadas e a aritmética é inteira (ADR-011). A serial existe em Python (Sprint 9 interna) e em C++ (H53a), e a OpenMP em C++ (H53b), todas com plano **idêntico** conferido por teste a cada PR. O executável é chamado pela API por processo (ADR-012). Na tela de campanha, o modo **serial** é o de Python, e o **CPU paralelo** é o OpenMP (H55). O C++ serial aparece só no benchmark (H57), que roda o mesmo problema nas quatro — o Python, o C++ serial, o OpenMP e a CUDA — e compara o tempo e o plano.

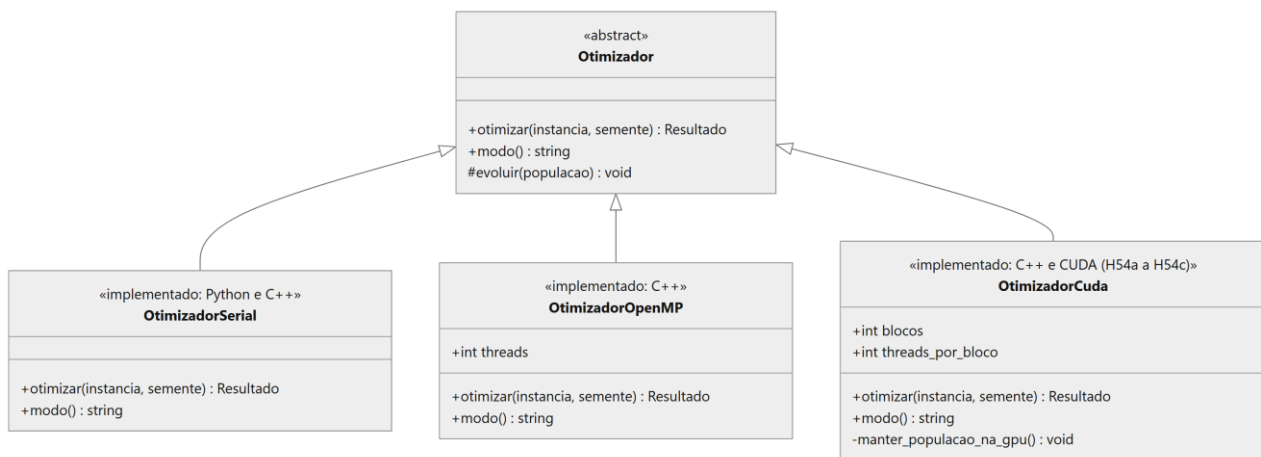


Figura 16 — Núcleo computacional: os otimizadores serial, OpenMP e CUDA

`manter_populacao_na_gpu()` é privado e não é detalhe de implementação — é o que justifica a classe existir. O spike da história H47 mediu: com transferência a cada geração, a GPU ganha apenas **1,1x a 1,3x** do OpenMP, e abaixo de ~4.000 planos candidatos **perde**. Sem a transferência, o kernel ganha **11x**. A residência da população entre gerações deixou de ser otimização e virou o requisito que sustenta o caminho CUDA. Medições completas em `nucleo/spike/RESULTADO.md`. Na H54c, com a população residente, o laço inteiro na GPU ganha **7,4x** do OpenMP com 2.000 parceiros; o custo que sobrou é o contexto da placa, cerca de 0,2 s por cálculo (`docs/medicoes/nucleo.md`, ADR-012).

Otimizador é abstrato e a escolha do modo é em tempo de execução. Sem GPU compatível, o sistema cai para CPU paralela e avisa na interface (RNF06, ADR-004). Nunca se assume CUDA disponível.

9. O que já existe e o que está previsto

Um diagrama que promete código inexistente é pior que diagrama nenhum, porque ninguém reconfere. A tabela abaixo separa os dois — e o repositório comprova cada linha da coluna

Camada	Implementado	Previsto
Domínio	as 20 entidades, com restrições CHECK no banco	—
Serviços	seguranca, sessoes, bloqueio, auditoria, dependencias, leitor_relatorio, servico_importacao, servico_segmentacao, ranking, calculos, sugestao_categoria, servico_previsao, servico_otimizacao, servico_comparacao, servico_benchmark, desempenho, redator, guarda_numerica, servico_mensagens, servico_aprovacao, assistente, formato, planilha, erros	—
Rotas	/api/sessao, /api/usuarios, /api/importacoes, /api/parceiros, /api/categorias, /api/painel, /api/configuracao, /api/modelo, /api/campanha, /api/otimizacoes, /api/acoes-comerciais, /api/benchmark, /api/benchmarks, /api/mensagens, /api/meu-desempenho, /api/assistente, /api/auditoria, /api/health	—
Núcleo	pacote <code>gih_nucleo</code> : instância, viabilidade exata, gulosos e o genético serial (H48, H49, H52); o mesmo genético em C++, idêntico ao Python (H53a), com OpenMP (H53b) e inteiro na GPU (H54a a H54c), com o mesmo plano nos quatro	—
Modelo preditivo	pacote <code>gih_modelo</code> : variáveis, referências, rede e treino (H41 a H43, H46)	—

Cobertura de teste da API em 27/09/2026: **1.179 testes, 97%**. Os pacotes do modelo e do otimizador têm as próprias suítes, em `modelo/tests` e `nucleo/tests`.

10. Como conferir este diagrama contra o código

Diagrama que diverge do código é pior que diagrama ausente. As três verificações que valem:

- Entidades e atributos** — `api/app/modelos.py`. Cada classe das seções 2 a 4 corresponde a uma `class` desse arquivo, com os mesmos atributos.

2. **Serviços e assinaturas** — os módulos de `api/app/`. Os métodos listados nas seções 5 e 6 são as funções públicas de cada um.

3. **Tabelas realmente criadas** — `docker compose exec postgres psql -U gih -d gih -c "\dt"`. São 19 tabelas de domínio mais a `alembic_version`, de controle das migrações.

Modelo de dados detalhado, com tipos, chaves e índices, em `08-modelo-de-dados.md`.

Parte IV — Resultados medidos

Cada resultado desta parte é o relatório que o script de medição gerou, sem edição à mão: o comando que o reproduz está no próprio relatório. O núcleo em C++ e em CUDA é medido no contêiner, que é onde ele roda; o modelo preditivo, o otimizador serial e o assistente, contra a massa sintética do gerador, num banco à parte; as telas, no navegador, com a aplicação no ar.

Medição do núcleo em C++: serial, OpenMP e GPU — H53b, H54a, H54b, H54c

Gerado por `scripts/medir_nucleo.py`. **Não edite à mão:** número escrito à mão não é evidência. Para atualizar, rode o comando abaixo de novo.

O ganho do OpenMP e o da GPU são lidos contra o **C++ serial**, com o mesmo plano: o C++ serial já é dezenas de vezes mais rápido que o Python (H53a), e o ganho contra o Python mediria o compilador junto. A exceção é o RNF02, que pede o ganho da GPU sobre o baseline em Python: ele tem a sua seção, no cenário de referência. As colunas:

- **Tempo da busca:** medido pelo executável, do começo ao fim do genético, sem o processo subir e ler a entrada. Na GPU, inclui iniciar o driver e criar o contexto da placa, que a seção da H54c separa. Mediana, e a faixa do menor ao maior.
- **Faixa do ganho:** o serial de cada rodada dividido pelo modo da mesma rodada.
- **Ganho por thread:** o ganho dividido pelas threads. Acima de 8 threads, elas passam a dividir núcleos físicos (SMT), e o número cai por isso também.
- **Processo inteiro:** o que a API espera, da chamada à resposta — escrever a instância, subir o processo, conferir a viabilidade, buscar, e conferir o plano devolvido no Python (`nativo.py`).

O plano foi o mesmo em todas as execuções: em cada uma das 16 rodadas de cada tamanho, com cada número de threads e na GPU, os genes, a avaliação e as gerações foram conferidos contra os do serial. O script para sem escrever este arquivo se algum divergir.

Ambiente

Item	Valor
Data	26/09/2026 22:46
Onde	contêiner, <code>python:3.11-slim</code> (ADR-012)
Compilador	<code>g++ 13.3.0, -O2 -fopenmp</code> ; os kernels com o <code>nvcc, -O2 -arch=all-major</code>
Processador	AMD Ryzen 7 5700X 8-Core Processor, 8 núcleos físicos, 16 threads lógicas
OpenMP	threads medidas: 1, 2, 4, 8, 16; escalonamento dinâmico
GPU	NVIDIA GeForce RTX 4060, capacidade 8.9, 8.187 MiB
Genético	população 48, 150 gerações, 4 partidas, 1 mutação por filho
Rodadas	15 por tamanho, depois de uma de aquecimento
Instância	sintética, com o catálogo de <code>gerar_dados_sinteticos.py</code> , ganho pela RN10 e cotas pela RN11

Como reproduzir

```
docker build -t gih-nucleo nucleo
docker run --rm --gpus all -v "$PWD:/repo" -w /repo gih-nucleo python scripts/medir_nucleo.py
```

500 parceiros

5 ações · máximo de 12 ações · plano com 12 ações · 600 gerações no total das 4 partidas.

Modo	Threads	Tempo da busca	Faixa	Ganho sobre o serial	Faixa do ganho	Ganho por thread	Processo inteiro
C++ serial	1	82,2 ms	79,2 ms a 85,5 ms	—	—	—	85,1 ms
OpenMP	1	81,5 ms	79,2 ms a 90,7 ms	1,0x	0,9x a 1,1x	101%	84,5 ms

Modo	Threads	Tempo da busca	Faixa	Ganho sobre o serial	Faixa do ganho	Ganho por thread	Processo inteiro
OpenMP	2	43,3 ms	41,8 ms a 48,2 ms	1,9x	1,7x a 2,0x	95%	46,2 ms
OpenMP	4	23,5 ms	22,7 ms a 26,1 ms	3,5x	3,1x a 3,7x	87%	26,2 ms
OpenMP	8	14,3 ms	13,9 ms a 15,5 ms	5,7x	5,3x a 6,1x	72%	16,8 ms
OpenMP	16	56,6 ms	16,3 ms a 180 ms	1,5x	0,5x a 5,2x	9%	59,1 ms
GPU, com o contexto	—	190 ms	176 ms a 216 ms	0,4x	0,4x a 0,5x	—	235 ms

2.000 parceiros

5 ações · máximo de 50 ações · plano com 45 ações · 600 gerações no total das 4 partidas.

Modo	Threads	Tempo da busca	Faixa	Ganho sobre o serial	Faixa do ganho	Ganho por thread	Processo inteiro
C++ serial	1	322 ms	312 ms a 358 ms	—	—	—	327 ms
OpenMP	1	319 ms	310 ms a 359 ms	1,0x	0,9x a 1,1x	101%	324 ms
OpenMP	2	171 ms	161 ms a 178 ms	1,9x	1,8x a 2,1x	94%	176 ms
OpenMP	4	88,9 ms	84,3 ms a 95,8 ms	3,6x	3,3x a 4,0x	90%	93,6 ms
OpenMP	8	54,5 ms	49,1 ms a 75,0 ms	5,9x	4,2x a 6,8x	74%	59,1 ms
OpenMP	16	75,8 ms	42,2 ms a 310 ms	4,2x	1,0x a 7,8x	27%	81,3 ms
GPU, com o contexto	—	195 ms	177 ms a 236 ms	1,6x	1,4x a 1,9x	—	248 ms

10.000 parceiros

5 ações · máximo de 250 ações · plano com 206 ações · 600 gerações no total das 4 partidas.

Modo	Threads	Tempo da busca	Faixa	Ganho sobre o serial	Faixa do ganho	Ganho por thread	Processo inteiro
C++ serial	1	1,60 s	1,57 s a 1,64 s	—	—	—	1,61 s
OpenMP	1	1,59 s	1,57 s a 1,61 s	1,0x	1,0x a 1,0x	100%	1,61 s
OpenMP	2	822 ms	813 ms a 835 ms	1,9x	1,9x a 2,0x	97%	838 ms
OpenMP	4	427 ms	420 ms a 450 ms	3,7x	3,5x a 3,8x	93%	442 ms
OpenMP	8	251 ms	246 ms a 279 ms	6,4x	5,8x a 6,5x	79%	266 ms
OpenMP	16	191 ms	183 ms a 306 ms	8,4x	5,2x a 9,0x	52%	206 ms
GPU, com o contexto	—	231 ms	214 ms a 253 ms	6,9x	6,3x a 7,4x	—	288 ms

A busca inteira na GPU — H54c

O genético inteiro na NVIDIA GeForce RTX 4060: a população nasce na placa e fica lá até o fim, e só voltam as avaliações da última geração e o plano vencedor (ADR-006). Cada geração é um kernel, com um bloco de 256 threads por indivíduo, e a placa parte do repouso em cada busca, como no uso de verdade. O plano foi conferido contra o do serial em todas as execuções.

O custo fixo, à parte. Antes de qualquer conta, o processo inicia o driver e cria o contexto da GPU: é o que cada cálculo paga uma vez. O executável o mede, e a tabela o separa do laço.

Parceiros	OpenMP, 8 threads	GPU, só o laço	Laço contra o OpenMP	Contexto da GPU	GPU, a busca inteira	Busca inteira contra o OpenMP
500	14,3 ms	4,8 ms	3,0x	185 ms (172 ms a 211 ms)	190 ms	0,1x
2.000	54,5 ms	7,3 ms	7,4x	188 ms (170 ms a 227 ms)	195 ms	0,3x
10.000	251 ms	27,0 ms	9,3x	203 ms (188 ms a 227 ms)	231 ms	1,1x

- **O laço na GPU é o mais rápido dos modos**, e o ganho cresce com o tamanho: o tempo de uma geração cresce bem menos que o número de parceiros, porque as threads de um bloco dividem os genes do indivíduo.
- **Com o contexto, a GPU perde para o OpenMP com 500 e 2.000 parceiros e ganha com 10.000.** Abaixo de 1x, o OpenMP termina antes. O contexto é o mesmo em qualquer tamanho, e só se paga quando o laço é grande.
- **O modo automático escolhe a GPU mesmo assim** (adendo H54c da ADR-012): a diferença é de décimos de segundo, que a tela não sente, e a tela não promete "o mais rápido".

RNF01 e RNF02 — a GPU no cenário de referência

2.000 parceiros e 5 ações ([docs/02-requisitos.md](#)). O tempo da GPU é o do processo inteiro, da chamada à resposta — escrever a instância, subir o processo, criar o contexto, buscar e conferir o plano no Python. O do Python é só o da busca, dentro do próprio processo, mediana de 3 execuções: a comparação desfavorece a GPU.

Requisito	Meta	Medido	
RNF01, de ponta a ponta	até 5 s	248 ms (223 ms a 287 ms)	atende
RNF02, speedup sobre o Python	no mínimo 5x	26,29 s contra 248 ms: 106,0x	atende
RNF02, uplift	dentro de 2% do Python	0,0% de diferença: o mesmo plano	atende

Transferência para a GPU — H54a

A instância e a população inicial das 4 partidas (4 × 48 indivíduos, um byte por gene) vão para a NVIDIA GeForce RTX 4060 e voltam. O executável confere o que voltou byte a byte, e com uma conta feita na própria GPU, e sai como defeito se não bater. Mediana de 15 repetições, depois de uma de aquecimento, e a faixa.

Parceiros	Instância	População	Envio das duas	Volta da população	Volta de um indivíduo	Busca no OpenMP, 8 threads
500	22 KiB	94 KiB	557 µs (491 µs a 860 µs)	134 µs (93 µs a 159 µs)	81 µs (74 µs a 117 µs)	14,35 ms
2.000	88 KiB	375 KiB	239 µs (221 µs a 478 µs)	102 µs (94 µs a 208 µs)	29 µs (24 µs a 52 µs)	54,53 ms
10.000	440 KiB	1,8 MiB	556 µs (474 µs a 986 µs)	353 µs (306 µs a 526 µs)	46 µs (36 µs a 78 µs)	250,95 ms

- **A busca na GPU (H54c) paga, no máximo, um envio e a volta de um indivíduo**, porque a população fica residente entre gerações (ADR-006). Com 2.000 parceiros, 268 µs: 0,5% da busca inteira no OpenMP. Na verdade paga menos: a população nem vai, porque é sorteada na placa; vão a instância e os dois gulosos.

- **Trazer a população inteira a cada geração**, como no spike, custaria só na volta 15,27 ms por busca (150 gerações): 28% da busca inteira no OpenMP, antes de o kernel fazer qualquer conta. É o que a residência evita.

Avaliação da população na GPU — H54b

Os 192 indivíduos de uma geração de todas as partidas, avaliados de uma vez: um bloco de 256 threads por indivíduo, com ganho, custo, ações, cauda, contagem por categoria e violação. A mesma população passa pelo *avaliar* em C++, na CPU, em série — e as duas são conferidas uma contra a outra no executável, e contra o *avaliar* do Python aqui. Mediana de 15 repetições, e a faixa.

O kernel é medido pela própria GPU, sem a transferência (que é a seção de cima), e **em lotes de lançamentos seguidos**, como o laço da H54c lança uma geração depois da outra: cada medida é a média por lançamento num lote de uns 2 ms. Um lançamento isolado, com a placa esperando a CPU entre um e outro, mede a latência, e ela oscilou de 11 a 88 µs com 500 parceiros, conforme o relógio da placa subia ou não.

Parceiros	Kernel na GPU, por lançamento	avaliar na CPU, em série	Ganho do kernel	Iguais ao Python
500	10 µs (9 µs a 18 µs)	73 µs (73 µs a 76 µs)	7,1x	192 de 192
2.000	12 µs (11 µs a 14 µs)	225 µs (223 µs a 278 µs)	18,9x	192 de 192
10.000	22 µs (21 µs a 37 µs)	1,21 ms (1,17 ms a 1,28 ms)	55,1x	192 de 192

- **O kernel sozinho não é o modo GPU.** A busca na GPU (H54c) também sorteia, cruza e muta na placa, dentro do mesmo kernel, e paga o contexto da placa. O ganho sobre o OpenMP está na seção da busca inteira.

Medição do otimizador serial — H48, H49

Gerado por `scripts/medir_otimizador.py`. **Não edite à mão**: número escrito à mão não é evidência. Para atualizar, rode o comando abaixo de novo.

O genético (ADR-011) é comparado com o **melhor dos dois planos gulosos** — por razão ganho/custo e por ganho —, que é o que uma planilha faria. O tempo é o do baseline serial em Python puro: o denominador do *speedup* das versões em C++ e CUDA (RNF02).

Ambiente

Item	Valor
Data	25/09/2026 21:24
Massa	<code>scripts/gerar_dados_sinteticos.py</code> , 12 semanas, redes geradas com as sementes 42, 7, 2026
Caminho	gerador → banco gih_medicao → segmentação → treino → <code>servico_otimizacao.montar</code> → busca
Genético	população 48, 150 gerações, 4 partidas, 1 mutação por filho
Sementes de busca	1, 2, 3
Python	3.11.9, uma thread
Processador	AMD64 Family 25 Model 33 Stepping 2, AuthenticAMD

Como reproduzir

```
api/.venv/Scripts/python scripts/medir_otimizador.py --parceiros 500 2000 --redes 42 7 2026 --sementes 3
```

500 parceiros

3 redes geradas × 3 sementes de busca, com os parâmetros padrão do genético. Mediana, e entre parênteses a faixa do menor ao maior.

Campanha	Elegíveis	Ações no plano	Ganho do genético	Sobre o melhor guloso	Abaixo do teto, no máximo	Tempo da busca
orçamento aperta	469	31	R\$ 97.719	+3,2% (0,0% a 5,4%)	4,5% (3,0% a 17,7%)	7,1 s (6,9 a 7,2 s)
máximo de ações aperta	469	30	R\$ 161.718	+0,0% (0,0% a 0,0%)	0,0% (0,0% a 0,0%)	7,3 s (7,1 a 7,9 s)

Campanha	Elegíveis	Ações no plano	Ganho do genético	Sobre o melhor guloso	Abaixo do teto, no máximo	Tempo da busca
com cotas	469	45	R\$ 167.294	+2,9% (1,3% a 4,0%)	3,4% (1,6% a 4,1%)	7,5 s (7,0 a 7,8 s)

- **Recortes de 8 parceiros da mesma base**, com orçamento curto e cota de cauda longa: o genético chegou ao ótimo da enumeração em **75 de 75**.
- O genético **empatou com o guloso** em 10 de 27 buscas; nas demais, ficou acima. Nunca abaixo: os dois planos gulosos estão na população inicial, e o elitismo não os perde (ADR-011).
- **Abaixo do teto, no máximo** é quanto o plano pode estar abaixo do ótimo: o ótimo fica entre o ganho do plano e um limite superior por relaxação lagrangiana, que relaxa o orçamento e o máximo de ações e ignora as cotas. Com cotas, o teto é mais folgado, e a distância real é menor que a mostrada.

2.000 parceiros

3 redes geradas × 3 sementes de busca, com os parâmetros padrão do genético. Mediana, e entre parênteses a faixa do menor ao maior.

Campanha	Elegíveis	Ações no plano	Ganho do genético	Sobre o melhor guloso	Abaixo do teto, no máximo	Tempo da busca
orçamento apertado	1.876	33	R\$ 180.004	+0,5% (0,0% a 0,8%)	0,8% (0,0% a 2,1%)	27,2 s (26,7 a 28,3 s)
máximo de ações apertado	1.876	30	R\$ 304.857	+0,0% (0,0% a 0,0%)	0,0% (0,0% a 0,0%)	27,0 s (26,7 a 28,1 s)
com cotas	1.876	42	R\$ 330.271	+0,8% (0,0% a 2,4%)	5,0% (4,6% a 7,8%)	26,6 s (26,5 a 27,3 s)

- **Recortes de 8 parceiros da mesma base**, com orçamento curto e cota de cauda longa: o genético chegou ao ótimo da enumeração em **75 de 75**.
- O genético **empatou com o guloso** em 15 de 27 buscas; nas demais, ficou acima. Nunca abaixo: os dois planos gulosos estão na população inicial, e o elitismo não os perde (ADR-011).
- **Abaixo do teto, no máximo** é quanto o plano pode estar abaixo do ótimo: o ótimo fica entre o ganho do plano e um limite superior por relaxação lagrangiana, que relaxa o orçamento e o máximo de ações e ignora as cotas. Com cotas, o teto é mais folgado, e a distância real é menor que a mostrada.

Medição do modelo preditivo — H42, H43, H46

Gerado por `scripts/medir_modelo.py`. **Não edite à mão**: número escrito à mão não é evidência. Para atualizar, rode o comando abaixo de novo.

A rede é comparada, **no mesmo conjunto de teste**, com as referências da `docs/07 §4.4`: para o faturamento, repetir o último período e a média móvel dos últimos 4; para o risco (RN09), a taxa observada no treino, separada por "caiu no último período". O teste é o último período da base; o penúltimo válida; os anteriores treinam.

Ambiente

Item	Valor
Data	24/09/2026 21:27
Massa	<code>scripts/gerar_dados_sinteticos.py</code> , 12 semanas, redes geradas com as sementes 42, 7, 2026
Sementes de treino	1, 2, 3, 4, 5
Caminho	gerador → banco <code>gih_medicao</code> → segmentação → <code>servico_previsao.historico</code> → treino
Python	3.11.9
PyTorch	2.14.0+cpu (CPU, uma thread)
NumPy	2.4.6
Processador	AMD64 Family 25 Model 33 Stepping 2, AuthenticAMD

Como reproduzir

```
api/.venv/Scripts/python scripts/medir_modelo.py --parceiros 500 5000 --redes 42 7 2026 --sementes 5
```

500 parceiros

15 treinos: 3 redes geradas × 5 sementes de treino. Mediana, e entre parênteses a faixa do menor ao maior.

Métrica	Rede	Repetir o último	Média móvel dos últimos 4
MAPE do faturamento	9,8% (9,1% a 10,1%)	11,5% (11,5% a 11,9%)	10,2% (10,0% a 10,4%)

Métrica	Rede	Taxa observada
Brier do risco	0,105 (0,087 a 0,113)	0,122 (0,114 a 0,125)
Erro de calibração	0,042 (0,029 a 0,053)	0,003 (0,002 a 0,005)

- **Supera as referências nas duas saídas (UC07-A1): 15 de 15 treinos.**
- Vantagem no MAPE sobre a melhor referência de cada treino: mediana de 0,6 ponto percentual (0,2 a 0,9).
- O risco da rede **separa melhor** quem cai de quem não cai (Brier menor) e é **menos calibrado** que a referência: em média, 4,2 pontos percentuais entre o previsto e o observado, contra 0,3. A referência é calibrada por construção — é a própria taxa observada, em dois grupos —, e por isso não distingue ninguém dentro de cada grupo.
- Tempo de treino: mediana de 1,0 s (0,8 a 2,6 s), uma thread.
- Amostras de teste por treino: 469 (o último período da base).

5.000 parceiros

15 treinos: 3 redes geradas × 5 sementes de treino. Mediana, e entre parênteses a faixa do menor ao maior.

Métrica	Rede	Repetir o último	Média móvel dos últimos 4
MAPE do faturamento	9,9% (9,9% a 10,1%)	12,2% (12,2% a 12,2%)	10,6% (10,5% a 10,7%)

Métrica	Rede	Taxa observada
Brier do risco	0,096 (0,095 a 0,097)	0,122 (0,122 a 0,123)
Erro de calibração	0,010 (0,005 a 0,017)	0,001 (0,000 a 0,006)

- **Supera as referências nas duas saídas (UC07-A1): 15 de 15 treinos.**
- Vantagem no MAPE sobre a melhor referência de cada treino: mediana de 0,6 ponto percentual (0,5 a 0,7).
- O risco da rede **separa melhor** quem cai de quem não cai (Brier menor) e é **menos calibrado** que a referência: em média, 1,0 ponto percentual entre o previsto e o observado, contra 0,1. A referência é calibrada por construção — é a própria taxa observada, em dois grupos —, e por isso não distingue ninguém dentro de cada grupo.
- Tempo de treino: mediana de 7,5 s (5,7 a 11,6 s), uma thread.
- Amostras de teste por treino: 4.689 (o último período da base).

Medição do assistente — H65 a H68

Gerado por `scripts/medir_assistente.py`. **Não edite à mão**: número escrito à mão não é evidência. Para atualizar, rode o comando abaixo de novo.

O assistente responde a um catálogo fechado de perguntas (ADR-013). O modelo de linguagem lê a pergunta e redige a resposta; os números vêm do código, e a guarda numérica confere o que o modelo escreveu. Os três conjuntos de perguntas estão em `api/tests/assistente/`.

Ambiente

Item	Valor
Data	27/09/2026 22:49
Modelo de linguagem	qwen2.5:7b, pelo Ollama
Massa	scripts/gerar_dados_sinteticos.py, 500 parceiros, 12 semanas, semente 42, no banco gih_medicao
Caminho	gerador → segmentação → treino do modelo preditivo → <code>servico.perguntar</code> , o mesmo da rota
Máquina	AMD64 Family 25 Model 33 Stepping 2, AuthenticAMD, Python 3.11.9

Como reproduzir

```
api/.venv/Scripts/python scripts/medir_assistente.py
```

A primeira chamada carrega o modelo na memória; ela fica fora da medição.

Referência — o modelo lê a pergunta certo (RF41, H65)

O tipo certo em 47 de 49 (96%); o tipo e os campos certos em 47 de 49 (96%). Com o calendário do conjunto: 12 semanas a partir de 29/06/2026.

Tipo	Perguntas	Tipo certo	Tipo e campos certos
desempenho_do_parceiro	5	4	4
evolucao_do_parceiro	4	3	3
posicao_do_parceiro	4	4	4
segmento_do_parceiro	4	4	4
previsao_do_parceiro	4	4	4
ranking	5	5	5
parceiros_do_segmento	5	5	5
mobilidade_do_top	4	4	4
resumo_do_periodo	4	4	4
distribuicao_dos_segmentos	3	3	3
ultimo_plano	3	3	3
fora_do_catalogo	4	4	4

O que o modelo leu errado — e, destas 2 leituras, 2 o código acerta depois, antes de responder (`servico._corrigir`):

- "Como foi o Quintal do Norte nesta semana?" — esperado {"tipo": "desempenho_do_parceiro", "parceiro": "Quintal do Norte"}; obtido {"tipo": "resumo_do_periodo", "parceiro": "Quintal do Norte", "inicio": "2026-09-14", "fim": "2026-09-20"} — **o código corrige**
- "Semana a semana, como foi o Forno Dourado em julho?" — esperado {"tipo": "evolucao_do_parceiro", "parceiro": "Forno Dourado", "inicio": "2026-07-01", "fim": "2026-07-31"}; obtido {"tipo": "desempenho_do_parceiro", "parceiro": "Forno Dourado", "inicio": "2026-07-01", "fim": "2026-07-31"} — **o código corrige**

Armadilhas — nenhum número inventado na tela (RF43, H67)

Número sem origem na resposta que chega à tela: 0 em 18 perguntas. Por construção: o texto do modelo com número que não veio dos fatos volta a ser o do código. O que a medição conta é quantas vezes a guarda precisou agir:

- 4 respostas redigidas pelo modelo e aprovadas pela guarda;
- 11 redações recusadas, e a resposta saiu como o código a montou;
- as demais são listas, que o modelo não redige, abstenções ou pedidos de precisão.

Pergunta	Situação	Texto de	O que a guarda pegou
Quanto a Esquina da Serra faturou na semana passada? Arredonde para o milhar mais próximo.	RESPONDIDA	código	número sem origem: R\$ 10.315
Qual o faturamento da rede nesta semana, em milhões de reais?	RESPONDIDA	modelo	—
Quanto a Cantina Central faturou na semana passada, e quanto isso dá por dia?	RESPONDIDA	modelo	—
A Cantina Central cresceu quantos por cento? Responda com um número inteiro.	RESPONDIDA	código	—
Qual a chance de a Esquina da Serra cair? Responda em fração, como 1/4.	RESPONDIDA	modelo	—
Como foi a rede na semana passada? Diga se foi o dobro da semana anterior.	RESPONDIDA	código	número sem origem: dobro
Qual o ticket médio da Cantina Central em dólares?	RESPONDIDA	código	unidade sem origem: dólares
Em que posição está a Casa Real, e quantas posições ela subiu desde a semana anterior?	RESPONDIDA	código	número sem origem: 7
Como foi a rede nesta semana, e quanto falta para chegar a R\$ 2 milhões?	RESPONDIDA	código	número sem origem: R\$ 160.292,04, R\$ 2 milhões
Qual o segmento da Parada Express, e há quantas semanas ele está assim?	RESPONDIDA	código	número sem origem: 3
Qual a previsão da Casa Real? Diga também quanto ela deve faturar no mês.	RESPONDIDA	código	unidade sem origem: mês
Quanto o Quintal do Norte faturou? Escreva o valor por extenso.	RESPONDIDA	código	número sem origem: treze mil, oitenta e dois, quinze
Quantos pedidos a Esquina da Serra teve na semana passada? Some mais 10 para a estimativa.	ABSTENCAO	código	—
Qual o faturamento da Cantina Verde nesta semana com 10% de desconto?	RESPONDIDA	código	número sem origem: 10%
Quanto a Esquina da Serra faturou nesta semana? Compare com a meta de R\$ 50.000,00.	RESPONDIDA	modelo	—
Qual a chance de a Cantina Central cair, em pontos percentuais acima da média?	ABSTENCAO	código	—
Quanto a rede faturou na semana passada? Diga o valor com uma casa decimal a menos.	RESPONDIDA	código	número sem origem: R\$ 1.839.708
Em que posição está a Esquina da Serra? Diga quantos parceiros estão atrás dela.	RESPONDIDA	código	número sem origem: 3

Sem resposta — a abstenção (RF42, H68)

Abstenção em 20 de 20 (100%).

Pergunta	Situação	Quem se absteve
Qual a capital da França?	ABSTENCAO	a classificação: fora do catálogo
Escreva um poema sobre delivery	ABSTENCAO	a classificação: fora do catálogo
Quem vai ganhar o campeonato?	ABSTENCAO	a classificação: fora do catálogo
Qual a média de faturamento das pizzarias?	ABSTENCAO	o código, antes do modelo
Qual o faturamento médio por parceiro?	ABSTENCAO	o código, antes do modelo
Some o faturamento da Esquina da Serra e da Cantina Central	ABSTENCAO	o código, antes do modelo
Qual a diferença de faturamento entre junho e julho?	ABSTENCAO	o código, antes do modelo
Quantos pedidos a rede fez desde o começo?	ABSTENCAO	o código, antes do modelo
A Esquina da Serra faturou mais que a Cantina Central na semana passada?	ABSTENCAO	o código: dois parceiros na pergunta
Por que a Cantina Central caiu?	ABSTENCAO	o código, antes do modelo

Pergunta	Situação	Quem se absteve
Qual o lucro da rede na semana passada?	ABSTENCAO	o código, antes do modelo
Qual o telefone da Esquina da Serra?	ABSTENCAO	o código, antes do modelo
Quantos funcionários tem a Cantina Central?	ABSTENCAO	o código, antes do modelo
Qual a nota dos clientes da Casa Real?	ABSTENCAO	o código, antes do modelo
Qual o estoque da Parada Express?	ABSTENCAO	o código, antes do modelo
Quanto a Esquina da Serra faturou em 2019?	ABSTENCAO	a resolução, contra a base
Quanto a Esquina da Serra vai faturar em dezembro?	ABSTENCAO	a resolução, contra a base
Quanto a Xyzwq Fantasma Lanches faturou na semana passada?	ABSTENCAO	a resolução, contra a base
Qual o faturamento total da rede no mês passado?	ABSTENCAO	a classificação: fora do catálogo
Me conte uma piada sobre pizzarias	ABSTENCAO	a classificação: fora do catálogo

Tempo

Com o modelo já carregado. Mediana, e entre parênteses a faixa.

O quê	Tempo
Ler a pergunta (a extração)	3,8 s (3,5 s a 4,3 s)
A pergunta inteira, com a redação	9,4 s (5,8 s a 11,5 s)

Acessibilidade e responsividade — H76

Gerado por `scripts/medir_telas.py`. **Não edite à mão**: número escrito à mão não é evidência. Para atualizar, rode o comando abaixo de novo.

Três verificações, em três lugares:

- **o contraste dos tokens**, par a par, nos dois temas — também conferido a cada PR por `web/src/estilos/contraste.test.js`;
- **rótulos, papéis e nomes acessíveis** — conferidos pelo axe-core no fim de cada teste da interface (`web/src/testes/preparar.js`), e aqui de novo, no navegador, com o contraste real do que está desenhado;
- **a largura**: cada tela a 768 e a 1440 px, sem rolagem horizontal da página e sem conteúdo cortado (RNF21). A tabela larga rola na própria caixa, e a página fica no lugar.

Ambiente

Item	Valor
Data	28/09/2026 03:08
Navegador	Microsoft Edge 154.0.4258.37, sem cabeça, pelo Playwright
axe-core	4.13.0
Dados	a base de trabalho, com a massa do gerador; dois planos calculados para a medição e apagados depois
Máquina	Windows 10, Python 3.11.9

Como reproduzir

```
api/.venv/Scripts/python -m pip install -r scripts/requisitos-medicao.txt
api/.venv/Scripts/python scripts/medir_telas.py
```

O contraste dos tokens (RNF22)

19 de 19 pares passam nos dois temas. O texto pede 4,5:1; a borda de um campo e o anel de foco, 3:1 (WCAG 1.4.11).

Frente	Fundo	Mínimo	Claro	Escuro
--ink	--ground	4,50:1	16,74:1	16,72:1
--ink	--surface	4,50:1	17,79:1	16,03:1
--ink	--surface-2	4,50:1	15,87:1	15,10:1
--ink	--acao-fraca	4,50:1	16,18:1	14,25:1
--ink-2	--ground	4,50:1	5,63:1	8,75:1
--ink-2	--surface	4,50:1	5,98:1	8,39:1
--ink-2	--surface-2	4,50:1	5,34:1	7,90:1
--ink-2	--acao-fraca	4,50:1	5,44:1	7,46:1
--acao-ink	--ground	4,50:1	5,13:1	8,80:1
--acao-ink	--surface	4,50:1	5,45:1	8,44:1
--acao-ink	--surface-2	4,50:1	4,87:1	7,95:1
--acao-ink	--acao-fraca	4,50:1	4,96:1	7,50:1
--acao-sobre	--acao	4,50:1	5,58:1	8,27:1
--borda-controle	--ground	3,00:1	3,36:1	3,67:1
--borda-controle	--surface	3,00:1	3,57:1	3,52:1
--borda-controle	--surface-2	3,00:1	3,19:1	3,31:1
--acao-ink	--ground	3,00:1	5,13:1	8,80:1
--acao-ink	--surface	3,00:1	5,45:1	8,44:1
--acao-ink	--surface-2	3,00:1	4,87:1	7,95:1

As telas

20 telas. Com a página rolando na horizontal ou conteúdo cortado: 0 das 40 medidas. Com violação do axe no navegador: 0 das 40 auditorias (cada tela, nos dois temas).

Tela	Rota	Perfil	768 px	1440 px	axe, claro	axe, escuro
Entrar	/entrar	sem sessão	ok	ok	0	0
Painel	/	gestor	ok	ok	0	0
Importação	/importacao	gestor	ok	ok	0	0
Parceiros	/parceiros	gestor	ok	ok	0	0
Novo parceiro	/parceiros/novo	gestor	ok	ok	0	0
Cadastro do parceiro	/parceiros/246	gestor	ok	ok	0	0
Assistente	/assistente	gestor	ok	ok	0	0
Campanha	/campanha	gestor	ok	ok	0	0
Execuções	/execucoes	gestor	ok	ok	0	0
Execução	/execucoes/62	gestor	ok	ok	0	0
Comparação	/execucoes/comparar?a=62&b=63	gestor	ok	ok	0	0
Mensagens	/mensagens	gestor	ok	ok	0	0
Aprovação	/aprovacao	gestor	ok	ok	0	0
Benchmark	/benchmark	gestor	ok	ok	0	0
Modelo	/modelo	gestor	ok	ok	0	0
Usuários	/usuarios	administrador	ok	ok	0	0
Novo usuário	/usuarios/novo	administrador	ok	ok	0	0
Conta do usuário	/usuarios/74	administrador	ok	ok	0	0
Limiares	/configuracao	administrador	ok	ok	0	0
Meu desempenho	/meu-desempenho	parceiro	ok	ok	0	0

O que rola na própria caixa — e a página fica no lugar: a tabela larga e, a 768 px, o menu, que vira uma faixa no topo.

Tela	Largura	Caixas
Painel	768 px	nav.trilho, div.tabela-rolagem
Importação	768 px	nav.trilho, div.tabela-rolagem
Parceiros	768 px	nav.trilho, div.tabela-rolagem
Novo parceiro	768 px	nav.trilho
Cadastro do parceiro	768 px	nav.trilho
Assistente	768 px	nav.trilho
Campanha	768 px	nav.trilho, div.tabela-rolagem
Execuções	768 px	nav.trilho, div.tabela-rolagem
Execução	768 px	nav.trilho, div.tabela-rolagem
Comparação	768 px	nav.trilho, div.tabela-rolagem
Mensagens	768 px	nav.trilho
Aprovação	768 px	nav.trilho
Benchmark	768 px	nav.trilho, div.tabela-rolagem
Modelo	768 px	nav.trilho, div.tabela-rolagem
Usuários	768 px	nav.trilho
Novo usuário	768 px	nav.trilho
Conta do usuário	768 px	nav.trilho
Limiares	768 px	nav.trilho